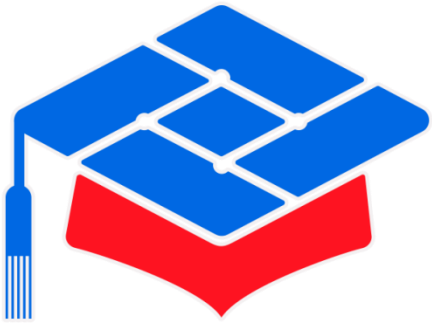


第十届

全国大学生集成电路创新创业大赛

报告类型：技术文档

参赛杯赛：“紫光同创”企业命题

作品名称：基于紫光 FPGA+RK3568 的视频流智能交通识别系统

队伍编号：CICC1001862

团队名称：ZLC

目录

作品预览简介 3
初赛方案 5
1. 系统组成及总体设计方案 5
1.1 整体介绍 5
1.2 硬件系统介绍 5
分赛区决赛方案 6
1. 系统组成及总体设计方案 6
1.1 整体介绍 6
1.2 硬件系统介绍 7
1.2.1 硬件整体介绍 7
1.2.2. 电路各模块介绍 7
总决赛方案 9
1. 系统组成及总体设计方案 9
1.1 整体介绍 9
1.2 算法介绍 10
1.2.1 算法整体介绍 10
1.2.2 RK3568 算法模块介绍 12
1.2.3. FPGA 车牌检测模型的训练与底层优化 31
1.2.4 FPGA 端算法模块介绍 36
2 性能参数 50
2.1 RK3568 侧误差测算 50
2.1.1. 模型训练概述 51
2.1.2 YOLO26 注意力模块改造 51
2.1.3 实验配置 51
2.1.4 模型的量化与部署 52
2.2. FPGA 资源开销 54
2.3. FPGA 车牌识别误差分析 54
3. 仿真与测试报告 55
3.1. 系统模块功能仿真测试 55
3.2 仿真环境与参数设置 55
3.3 模块仿真结果分析 56
4. 总结 59
4.1 可扩展之处 59
4.2 心得体会与小组分工 60
5 参考文献 61
6 AI 使用说明 61

作品预览简介

本项目设计了一套基于FPGA与ARM异构架构的智能交通视频流分析系统。系统采用盘古 PG2L200H-6IFBG676 与 RK3568 作为核心硬件平台，主要实现两大核心功能：**基于视频流的实时车牌检测与识别、基于视频流的行人闯红灯检测。**FPGA 负责 1920×1080@60 Hz HDMI 图像视频流采集，利用 7 层卷积和 2 个检测头组成的 9 层 YOLO 架构 CNN 完成车牌候选区域检测。FPGA 将最多 4 个候选框的轴对齐矩形坐标通过 UART 发送至 RK3568。RK3568 根据 FPGA 发送的候选 ROI 完成更高精度的车牌位置、四角点检测和字符识别，在此基础上我们也完成了另一套由 RK3568 独立执行完整的车牌检测与识别的方案。

关键技术：

- **FPGA 端 YOLO 架构车牌候选框检测与硬件加速：**针对 FPGA 资源和实时处理需求，设计并部署由 7 层骨干卷积和 2 个检测头组成的 9 层 YOLO 风格 INT8 CNN。网络输入为 320×192 RGB 图像，统一卷积每次处理 16 个输入通道，并行生成 8 个输出通道；每个输出通道由 4 个 dot4 乘加子单元组成，从而形成 32 个 dot4 子单元的分组 MAC 计算阵列。MAC 计算阵列结合特征缓存乒乓存储、INT32 累加和定点重量化，能够有效降低片上存储与计算资源开销。最后经过 Sigmoid、边界框解码、Top-K 和 NMS 模块筛选并输出车牌候选 ROI 的轴对齐矩形坐标。
- **RK3568 边缘 AI 部署与推理：**RK3568 端部署了 YOLO26 检测与 CTC 文字识别两个模型。软件上采用多线程架构，采集子线程负责读视频流，通过队列传给主推理线程做计算，两者互不阻塞。同时，视频解码和图像格式转换用 MPP 和 RGA 在硬件上完成，CPU 只负责推理逻辑，整体效率提升明显。
- **全栈远程管理系统：**采用前后端分离架构搭建 Web 远程管理平台。通过 SSH 和 WebSocket 协议，不仅支持对嵌入式端多种推理模式的远程启停控制，还能实现实时终端日志的低延迟推送与监控。

重要指标：

- **输入源与接口指标：**视频接口采用 MS7200 与 MS7210 构成的 HDMI 收发链路，兼容 HDMI 标准接口。工作于 **1920×1080@60 Hz** 模式，像素时钟为 148.5 MHz。支持 1920×1080@60 Hz 电脑、1080p@60Hz 摄像头输入。

- **系统吞吐与接口能力：**FPGA 最大端到端延时 **27.082 ms（首帧）**；单帧 CNN 计算时间为 **1,555,308 个计数周期**，约 **10.473 ms**；CNN 核心吞吐率为 **95.48 次/s**。RK3568 单独完成检测识别全流程端到端延时约 **30ms**；由 FPGA 发送的 ROI 坐标，RK 解析车牌区域进行字符识别端到端时延约 **10ms**。实际串口接收坐标实测 FPGA 车牌刷新时延为 **16.67ms**。
- **模型量化精度：**RK3568 模型算法模型经 RKNN-Toolkit2 进行 INT8 量化后，检测模型 mAP@0.5 达 0.935；识别模型整牌准确率达 0.936。FPGA 经 PTQ 量化导出 Precision 为 0.9386、Recall 为 0.9334、mAP@0.5 为 0.9666、mAP@0.5:0.95 为 0.6907。
- **推理加速性能：**部署至端侧后，检测推理单帧耗时为 20ms，识别推理单张车牌耗时为 1ms，充分满足智能交通场景的实时性要求。
- **支持识别功能与环境适应性：**FPGA 利用 YOLO 架构，支持蓝、黄、绿、黑、白色以及多车牌检测；容忍一定光线条件差、雨雪天气环境，容忍最大 30°倾斜。FPGA 协同 RK3568 支持蓝、黄、绿、黑、白色单个或多个车牌识别（最多 5 个），可进行特殊军警、领事馆车牌识别；容忍光线条件差、一定范围过曝、雨雪天气环境，容忍 30°以内倾斜。
- **车牌识别实时帧率：**通过串口发送最多 4 个车牌候选框的轴对齐矩形坐标、RK3568 定时解析坐标并进行当前帧识别，通过异步架构（显示识别分离框架）实时视频流显示帧率达 **60FPS**。

作品亮点

- **软硬异构协同：**系统提供多种运行模式。在 “串口坐标快速识别”模式中，FPGA 完成车牌候选区域的初步框选，并通过 UART 将 ROI 坐标发送至 RK3568。RK 端接收坐标后，先通过 YOLO26 模型进行坐标框内小范围的校验，再送入 OCR 模型完成车牌文字识别，从而兼顾处理速度与识别精度。
- **FPGA 侧 CNN 部署与硬件加速：**在 FPGA 上部署由 7 层骨干卷积和 2 个检测头组成的 9 层 INT8 **YOLO 架构** CNN，完成 HDMI 视频采集、6×6 降采样、320×192 帧缓存、CNN 推理、Top-K、NMS 及检测框叠加。统一卷积引擎采用 16 输入、8 输出通道并行结构，并通过双 Bank 缓存、Mailbox 和 frame_id 实现帧级同步。FPGA 输出最多 4 个车牌矩形候选框。

初赛方案

1.系统组成及总体设计方案

1.1 整体介绍

系统采用 FPGA+Arm 异构思路，采用盘古 PG2L200H-6IFBG676+RK3568 的硬件平台，核心实现两大功能：**车牌识别**和**行人闯红灯检测。**基于模块化设计思想，由 FPGA 负责高速图像预处理与实时车牌检测及行人闯红灯检测；RK3568 可支持完整的车牌检测与识别流程；同时可通过串口传输 FPGA 已定位的车牌坐标实现 RK 车牌识别。通过并行处理架构实现高效的视频流分析。核心部分包括：FPGA 端图像采集与预处理模块及车牌框选、RK3568 车牌检测与识别部分、行人闯红灯两种方法实现、DDR3 内存模块、输出显示模块、以太网和 USB 的视频输出接口。

1.2 硬件系统介绍

系统使用盘古 PG2L200H-6IFBG676[1]开发板，摄像头 OV5640[2]通过 FMC[3]扩展板与 FPGA 相连，摄像头采集车牌照片、行人闯红灯视频支持 DDR 读取在 FPGA 硬件[4]流水线处理完成车牌框选、行人闯红灯框定并输出 HDMI 显示；或经过以太网、HDMI-USB 转接口传入 RK3568 通过 NPU 单独完成高精度字符识别与行人检测；同时支持通过串口从 FPGA 向 RK 传输实时车牌坐标，帮助 RK 跳过定位算法加速 RK 识别。

分赛区决赛方案

1.系统组成及总体设计方案

1.1 整体介绍

系统仍采用 FPGA+Arm 异构思路，基于盘古 PG2L200H-6IFBG676+RK3568 的硬件平台，核心实现两大功能：**车牌识别**和**行人闯红灯检测**。在初赛方案的基础中，我们发现形态学准确率欠佳，受环境光线等因素影响较大，我们学习并部署成功了 FPGA 侧输入尺寸为 **128×128 的 6 层 CNN**、**320×320 的 23 层 CNN**。在初赛基础上扩展输入源，融合 **1920×1080，60Hz 的 HDMI 输入**，支持电脑屏幕、1080p 摄像头。全流程由 FPGA 利用 CNN 卷积计算负责车牌检测框选，RK3568 可支持完整的车牌检测与识别流程；同时可通过串口传输 FPGA 已定位的车牌坐标实现 RK 车牌识别，通过并行处理架构实现高效的视频流分析。行人闯红灯复用初赛方案，分别在 FPGA 和 RK3568 保留两种不同实现。

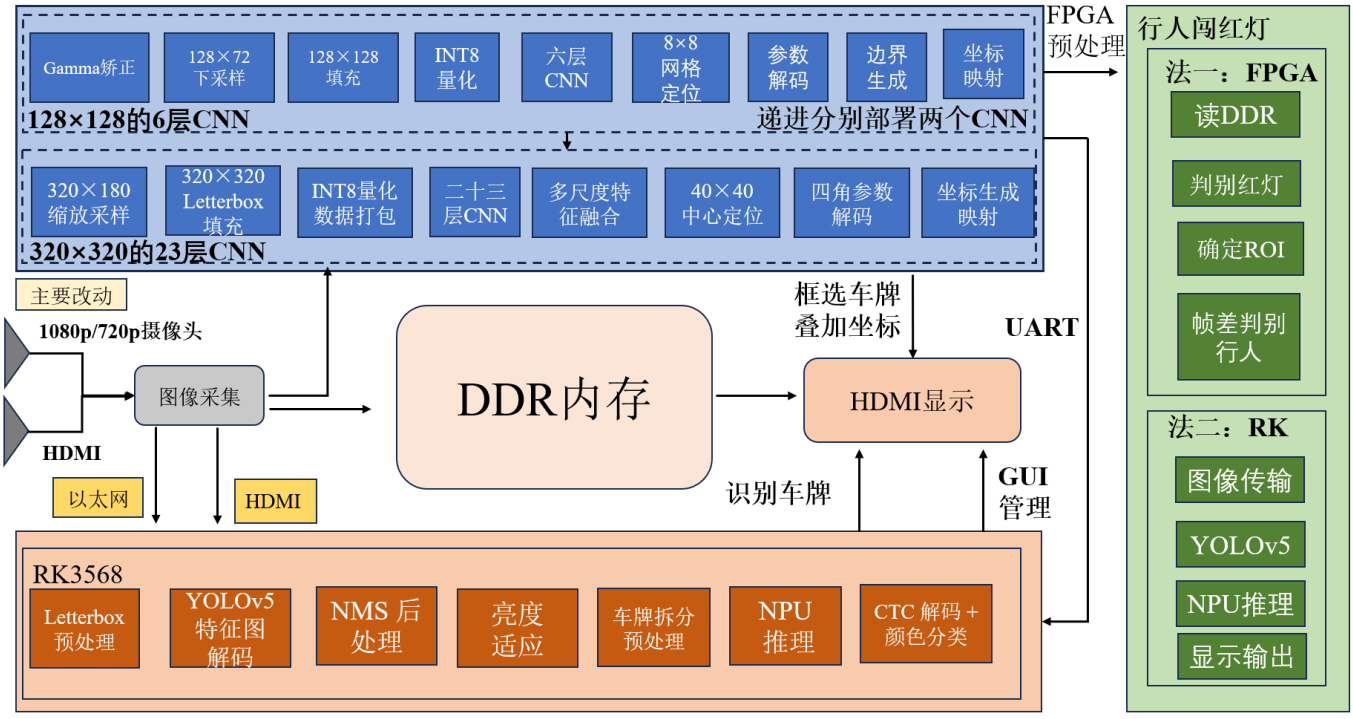


图 1 改进系统方案流程图

1.2 硬件系统介绍

1.2.1 硬件整体介绍

系统使用盘古 PG2L200H-6IFBG676[1]开发板，在初赛硬件上扩展 1080p 输入，支持 1080p 摄像头和电脑屏幕 1080p 作为输入源。输入源采集车牌照片、行人闯红灯视频支持 DDR 读取并在 FPGA 硬件[4]流水线处理完成车牌框选、行人闯红灯框定并输出 HDMI 显示；或经过 HDMI-USB 转接口传入 RK3568 通过 NPU 单独完成高精度字符识别与行人检测；同时支持通过串口从 FPGA 向 RK 传输实时车牌坐标，帮助 RK 舍弃检测定位算法加速 RK 识别。

硬件接口示意如下：

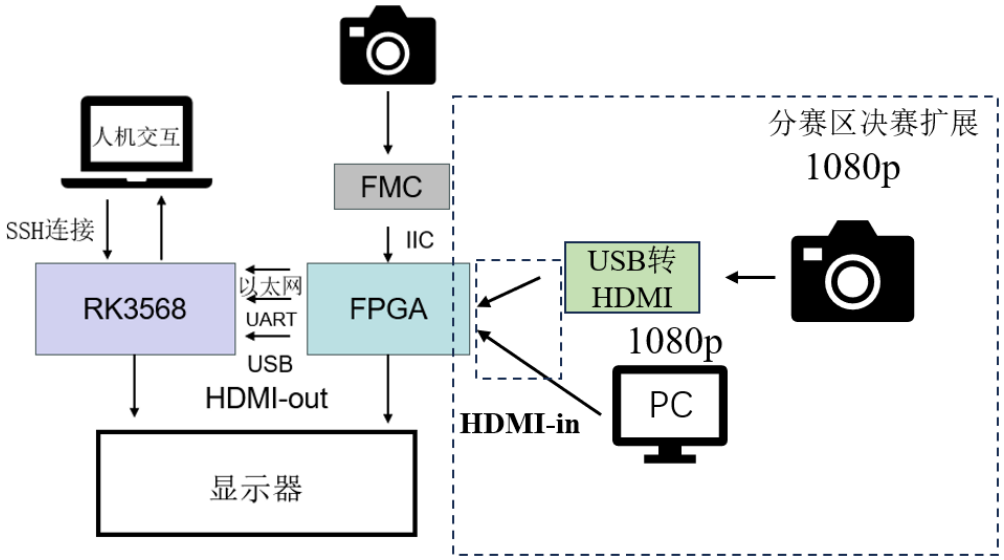


图 2 硬件系统接口

1.2.2. 电路各模块介绍

1.UART 电路：

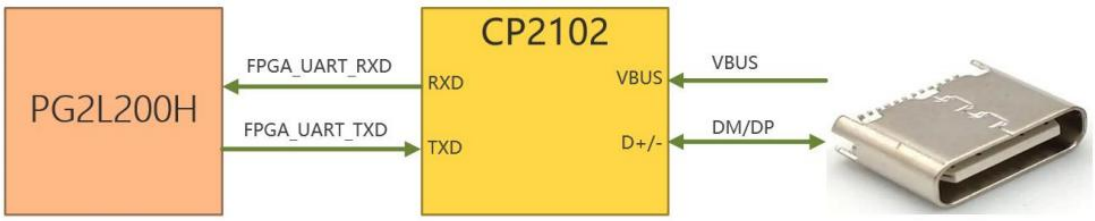


图 3 UART 电路图

2.HDMI 电路：

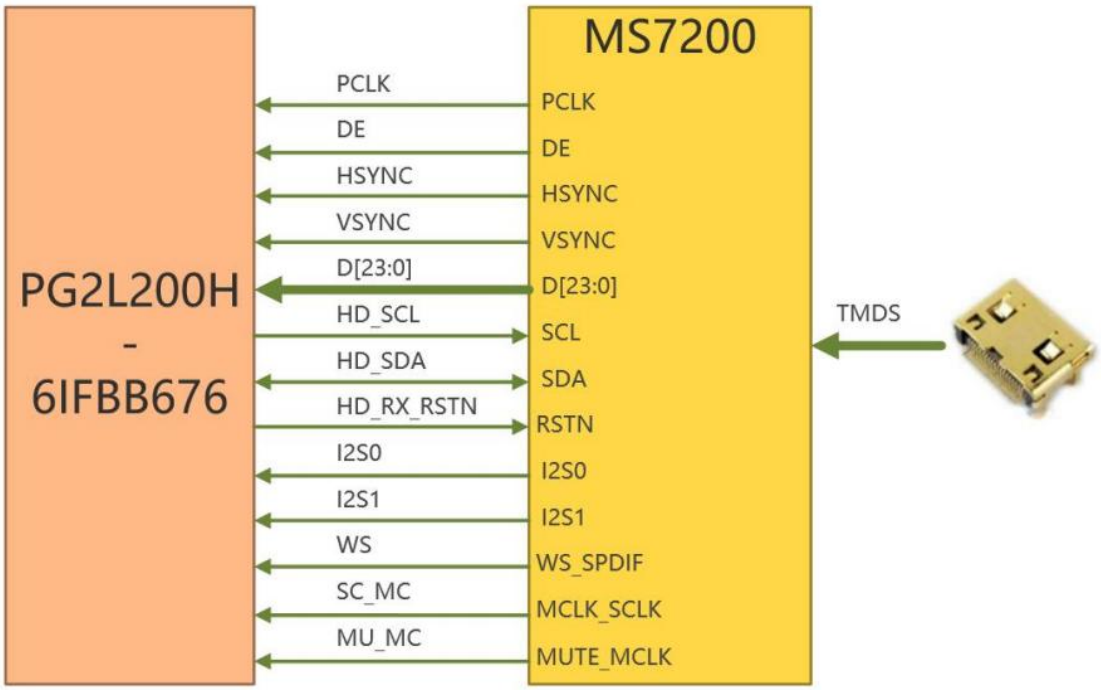


图 4 HDMI 电路图

3.HDMI 转 USB 转接模块电路图：

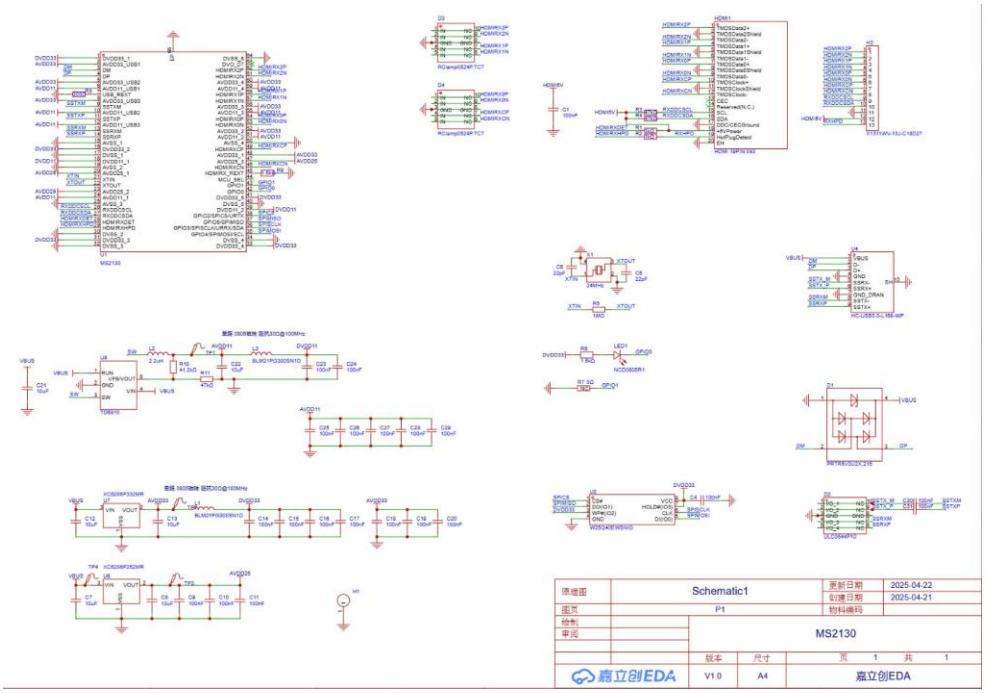


图 5 HDMI 转 USB 电路图

4.USB 转 HDMI 转换模块电路图：（部分）

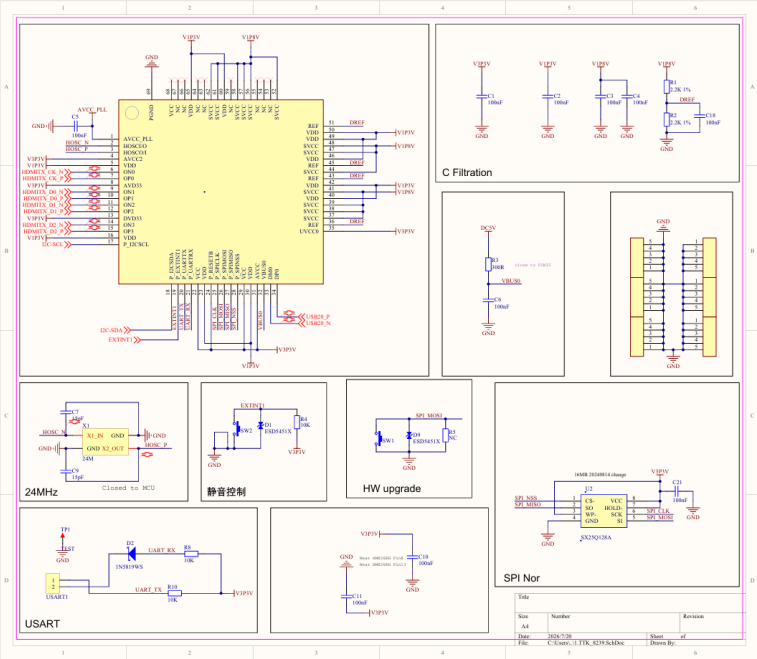


图 6 USB 转 HDMI 模块电路图

总决赛方案

1. 系统组成及总体设计方案

1.1 整体介绍

系统在分赛区决赛架构的基础上继续优化，核心实现车牌识别和行人闯红灯检测两大功能。针对 FPGA 侧原有 CNN 对小尺寸、倾斜及复杂场景车牌检测能力有限的问题，系统设计并部署了面向 FPGA 的 YOLO 风格车牌检测网络。该网络以 320×192 为输入，采用 8 个输出通道并行、每通道 16 路 UINT8×INT8 乘累加的计算结构，按照固定顺序串行执行 7 层骨干卷积和 2 个检测头，在 80×48 和 40×24 两个尺度上生成检测结果。检测输出经过 Sigmoid 查表、锚框解码、置信度过滤、Top-32 排序和 NMS 处理后，最多保留 4 个候选 ROI，并将检测框映射回 1920×1080 视频进行 HDMI 叠加显示。考虑到 FPGA 侧轻量化模型对车牌四角点的定位精度有限，决赛方案不再由 FPGA 直接输出四角点坐标，而是仅输出候选 ROI 坐标，再由 RK3568 端的 YOLO26 模型进一步检测车牌位置和四角点，从而提高小尺寸、倾斜及复杂场景下的定位精度。系统继续支持 1920×1080@60Hz HDMI 输入，可接入电脑画面和 1080p 摄像头。RK3568 既可接收 FPGA 通过串口发送的 ROI 坐标并完成后续检测与识别，也可独立完成完

整的车牌检测和识别流程。行人闯红灯功能沿用原有方案，在 FPGA 和 RK3568 端保留相应实现。系统流程图如图 7 所示。

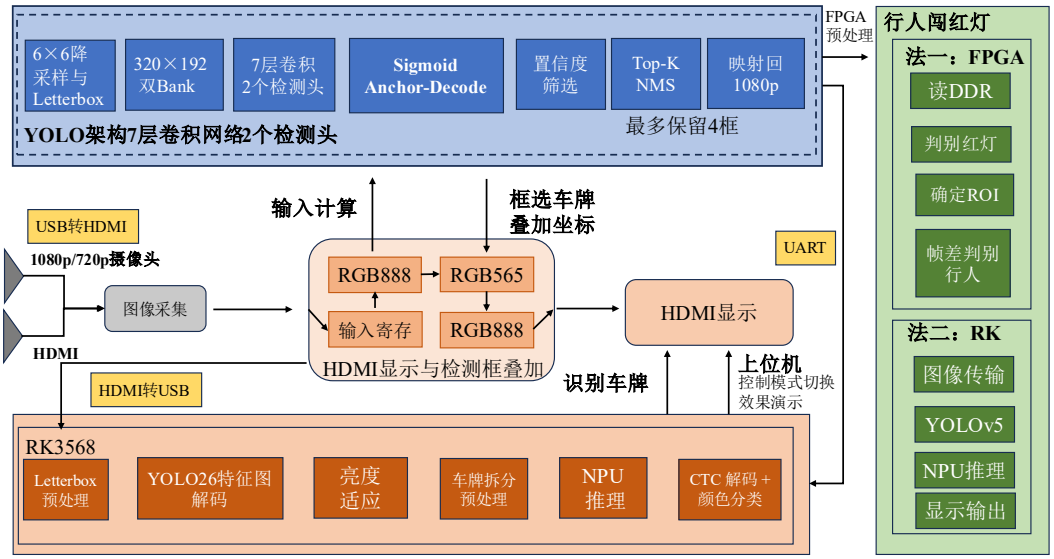


图 7 系统流程图

1.2 算法介绍

1.2.1 算法整体介绍

FPGA 端：系统以 HDMI 输入的 1920×1080 RGB888 图像为数据源，通过 6×6 面积降采样获得 320×180 图像，并在上下各填充 6 行 RGB 114 像素，形成 320×192 RGB888 网络输入。卷积计算单元采用 8 个输出通道并行结构，每个通道同时完成 16 组 UINT8 激活值与 INT8 权重的乘累加，并以 INT32 格式保存偏置和累加结果。网络按照固定调度顺序串行执行 7 层骨干卷积和 2 个检测头，在 80×48 和 40×24 两个尺度上输出检测结果。每个检测头包含 3 个锚框，每个锚框输出 4 个坐标参数、1 个目标置信度和 1 个类别分数，共 18 个 INT16 Q6.10 通道。检测结果随后经过 Sigmoid 查表、锚框解码、0.25 置信度过滤、Top-32 排序和 IoU 阈值为 0.45 的 NMS 处理，最多保留 4 个目标框，并将坐标映射回 1920×1080 图像进行 HDMI 画框显示，同时可通过 UART 发送检测框坐标。该网络保留了 YOLO 多尺度、单阶段检测机制，但省略了标准 YOLOv5 中的 CSP、SPPF 及 PAN/FPN 等复杂结构，并采用片上双缓冲 BRAM、固定参数 ROM 和定点计算，以降低 FPGA 资源占用和计算复杂度；当前推理与视频显示通路不使用外部 DDR3。

RK3568 端：采用零拷贝、全链路硬件加速与异步流水线架构，实现 1920×1080@60fps 视频的实时车牌检测、识别和显示，主要技术如下：

1. **零拷贝视频处理链路**：通过 V4L2 采集 MJPEG 视频流，使用 MPP 硬件解码为 NV12 帧，并由 Native 层统一维护缓冲区及其生命周期。NV12 帧直接交由 RGA 预处理和 DRM 显示，避免在 Python 层生成、传递和复制完整 BGR 图像，降低 CPU 占用与内存带宽消耗。
2. **硬件加速检测与识别**：RGA 直接对 NV12 帧执行缩放、letterbox 填充和颜色转换，生成 YOLO26 Pose 模型所需的 RGB NHWC 输入。NPU 检测车牌边界框、四角点和单双层类别；随后使用 OpenCV 完成透视校正、双层拼接及 168×48 BGR 缩放，并通过 Native OCR 模型识别车牌文字。
3. **异步双缓冲流水线**：视频采集、检测、识别和显示异步运行，并通过 latest-wins 队列优先处理最新数据。检测预处理采用双输入缓冲，使 RGA 准备下一次输入时，NPU 可同时处理当前输入，避免推理阻塞视频采集和显示。
4. **事件驱动与原生 C 调度**：主循环由视频帧、检测结果和识别结果等事件唤醒，无任务时阻塞等待。采集、MPP 解码、NV12 帧管理、RGA 预处理、RKNN 推理、模型解码和 DRM 显示等性能关键链路均由 Native C 模块实现，Python 层主要负责任务编排和结果管理。
5. **DRM Atomic 显示与结果输出**：使用 DRM/KMS Atomic 接口分别提交视频平面和标注平面，避免对完整视频帧进行重复绘制与合成。视频画面连续输出，仅在检测或识别结果变化时更新 Overlay，从而减少 CPU 绘制、图像拷贝和显示提交开销，降低显示延迟并保持画面流畅稳定。

1.2.2 RK3568 算法模块介绍

术语	说明
RKNN	Rockchip NPU 推理引擎及开发工具包
NPU	神经网络处理单元
MPP	Rockchip 媒体硬件编解码平台
RGA	Rockchip 2D 图形加速器，负责缩放与色彩空间转换
DRM/KMS	Linux 直接渲染管理与内核模式设置显示框架
CTC	连接时序分类算法
NMS	非极大值抑制
V4L2	Linux 视频采集框架
MJPEG	Motion JPEG 视频编码格式
CTC 字典	78 字符车牌字符映射表

表 1 RK3568 关键术语介绍

分类	组件、类别	规格与说明
硬件环境	主控板	RK3568
	FPGA 板卡	PG2L200H-6IFBG676
	HDMI 采集卡	MACROSILICON 芯片，V4L2 接口
	HDMI 显示器	DRM/KMS 加 RGA 双缓冲输出
软件环境	操作系统	Linux
	编程语言	Python，C++
	算法和视觉库	RKNN-Toolkit2、OpenCV、NumPy
	硬件加速	MPP 硬件解码、RGA 预处理与色彩转换、DRM/KMS 显示
	硬件通信库	Pyserial
	前后端框架	Flask、Vue3、Element Plus

表 2 作品依赖环境

RK3568 架构

软件框架

```
final                                  RK3568 生产程序目录
├─ benchmark_pipeline.py               实时流水线性能测试
├─ core                                硬件与通用基础模块
```

```
│   ├── camera_fps.py                摄像头帧率参数配置
│   ├── display_drm_atomic.py        DRM 原子零拷贝显示
│   ├── display_drm_rga.py           RGA 辅助画面显示
│   ├── fpga_io.py                   FPGA 以太网通信模块
│   ├── hdmi_io.py                   HDMI 采集参数定义
│   ├── image_utils.py               图像缩放坐标工具
│   ├── latency_metrics.py           流水线时延统计工具
│   ├── native                       原生 C 库源码与产物
│   │   ├── librga_preprocess.so     RGA 预处理动态库
│   │   ├── librkdrm_atomic.so       DRM 原子显示动态库
│   │   ├── librkdrm_display.so      本地图像显示动态库
│   │   ├── librkhdmi_capture.so     HDMI 硬件采集动态库
│   │   ├── librknn_runtime.so       RKNN 推理运行动态库
│   │   ├── Makefile                 原生动态库构建脚本
│   │   ├── rga_preprocess.c         RGA 模型预处理实现
│   │   ├── rkdrm_atomic.c           DRM 原子显示实现
│   │   ├── rkdrm_display.c          本地图像显示实现
│   │   ├── rkhdmi_capture.c         HDMI 采集解码实现
│   │   └── rknn_runtime.c           RKNN 推理解码实现
│   ├── native_capture.py            原生采集接口封装
│   ├── publisher.py                 上位机数据发布模块
│   ├── realtime_pipeline.py         实时异步队列管理
│   ├── rga_preprocess.py            RGA 预处理接口封装
│   ├── rknn_native.py               Native RKNN 接口封装
│   ├── serial_io.py                 串口 ROI 协议解析
│   └── __init__.py                  核心模块包初始化
├── fpga_ai.py                       RK 独立车牌识别入口
├── fpga_serial.py                   FPGA 串口识别入口
├── people_det                       行人闯红灯检测模块
```

│ ├─ app.py	行人模式启动与配置
│ ├─ people_detector.py	行人模型检测与后处理
│ ├─ runtime.py	行人实时流水线实现
│ └─ __init__.py	行人模块包初始化
├─ people_redlight.py	行人闯红灯程序入口
├─ performance.sh	系统性能采集脚本
├─ plate_det	车牌检测识别模块
│ ├─ ai_runtime.py	RK 车牌模式运行时
│ ├─ detection_filter.py	低置信候选过滤器
│ ├─ model_contracts.py	模型张量合同校验
│ ├─ plate_pipeline.py	车牌透视预处理流程
│ ├─ plate_recognition.py	字符表与双层拼接
│ ├─ plate_recognizer.py	车牌 OCR 识别解码
│ ├─ result_utils.py	检测结果通用工具
│ ├─ serial_runtime.py	串口 ROI 模式运行时
│ └─ __init__.py	车牌模块包初始化
├─ rknn_models	RKNN 生产模型目录
│ ├─ people_redlight_yolov5_384x640_int8.rknn	行人闯红灯检测模型
│ ├─ plate_ai_pose26_512x288_int8.rknn	车牌角点检测模型
│ ├─ plate_ocr_168x48_int8.rknn	车牌字符识别模型
│ └─ plate_serial_roi_pose26_320x160_w8a8_fp16.rknn	串口 ROI 检测模型
└─ tools	性能测试辅助工具
├─ benchmark_rknn_npu.py	RKNN 纯 NPU 基准测试
├─ perf_metrics.py	性能日志解析统计
└─ __init__.py	工具模块包初始化

表 3 目录结构

嵌入式端 AI 应用部署在 RK3568 开发板的/home/cat/RK3568/final 目录。系统以 Python 负责业务编排，模型使用 RKNN-Toolkit2 工具链完成转换，并通过 RKNN Runtime 调用 RK3568 NPU 推理。针对实时视频处理需求，系统采用“应

用入口层、AI 检测识别层、核心基础层、硬件加速层”四层架构，将视频采集、硬件解码、图像预处理、NPU 推理和显示合成等性能关键路径下沉至 C 语言动态库，实现 1080p、60 帧视频的零拷贝异步处理。

应用入口层：提供三个独立的生产运行入口，每个入口对应一种应用场景。

fpga_ai.py：执行 RK3568 独立车牌检测与识别流程。系统接收 FPGA 视频流，依次调用 YOLO26 Pose 车牌检测模型和车牌 OCR 模型，并将识别结果叠加到 HDMI 画面。

fpga_serial.py：执行 FPGA 串口 ROI 车牌识别流程。FPGA 通过串口发送最多四个车牌的矩形 ROI 坐标，RK3568 将 ROI 组成四槽模型输入，经过 YOLO26 Pose 模型进一步检测车牌四角点及单双层类型，再完成透视校正和车牌识别。

people_redlight.py：执行行人闯红灯检测，支持 HDMI 采集卡和本地视频文件两种输入方式，可使用 Native C 或显式 Python Toolkit 后端完成模型推理。

AI 检测识别层：封装车牌识别和行人闯红灯检测的业务逻辑。

ai_runtime.py：负责 RK 独立模式的采集、检测、识别、显示和异常恢复编排。

serial_runtime.py：负责串口接收、四槽组图、Pose 检测、OCR 识别和结果发布。

model_contracts.py：校验 YOLO26 Pose 模型的输入、输出尺寸及量化参数。

plate_pipeline.py：完成车牌四点透视校正、双层车牌拼接和尺寸调整。

plate_recognizer.py：调用车牌 OCR 模型，完成 CTC 文字解码和内部颜色分类。

plate_recognition.py：保存车牌字符表、颜色类别及双层车牌拼接工具。

detection_filter.py：对低置信度检测候选进行连续帧确认和质量过滤。

result_utils.py：提供车牌结果完整性判断和边界框交并比计算。

people_detector.py：执行行人闯红灯模型推理、检测结果后处理和标注数据生成。

people_det/runtime.py：实现行人模式的异步采集、检测和显示流水线。

核心基础层：提供硬件接口、并发调度、结果发布和通用处理能力。

native_capture.py：封装 V4L2 采集、MPP 硬件解码及 NV12 DMA 帧管理接口。

rga_preprocess.py：封装 RGA 硬件预处理，包括图像缩放、letterbox 和 ROI 提取。

rknn_native.py：封装 RKNN 模型加载、NPU 推理、Pose 解码和车牌 OCR 接口。

display_drm_atomic.py：显示 NV12 视频，并使用 ARGB 图层叠加检测结果。

display_drm_rga.py：负责本地 BGR 视频的 RGA 转换和 DRM 显示。

realtime_pipeline.py：提供 latest-wins 队列、帧引用计数和异步任务调度。

latency_metrics.py：统计采集、解码、排队、NPU 推理、显示和端到端时延。

publisher.py：向上位机发布原始 MJPEG 视频、检测结果和运行状态。

camera_fps.py：配置 USB 采集设备的帧率、曝光和增益。

serial_io.py：解析 FPGA 通过 UART 发送的变长 ROI 坐标协议。

fpga_io.py：实现 FPGA 以太网通信和 RGB565 解码能力。

hdmi_io.py：保存 HDMI 采集设备、分辨率、帧率和视频格式等常量。

硬件加速层：使用 C 语言实现性能关键功能，并编译为五个生产动态库供 Python 调用。

rkhdmi_capture.c → librkhdmi_capture.so：负责 V4L2 视频采集、MPP MJPEG 硬件解码和 NV12 DMA-BUF 帧池管理。

rga_preprocess.c → librga_preprocess.so：使用 RGA 完成模型 letterbox 预处理和 NV12 车牌 ROI 提取。

rkdrm_atomic.c → librkdrm_atomic.so：负责 NV12 视频 Plane、ARGB 标注 Plane、Atomic Commit 和 Page Flip 显示。

rkdrm_display.c → librkdrm_display.so：负责本地 BGR 视频的 RGA 转换与 DRM 双缓冲显示。

rknn_runtime.c → librknn_runtime.so：负责 RKNN 模型加载、持久输入缓冲、NPU 推理、YOLO26 Pose 解码和 Native 车牌 OCR。

Makefile：配置 DRM、RGA、MPP、FreeType 和 RKNN Runtime 依赖，并编译生成五个生产动态库。

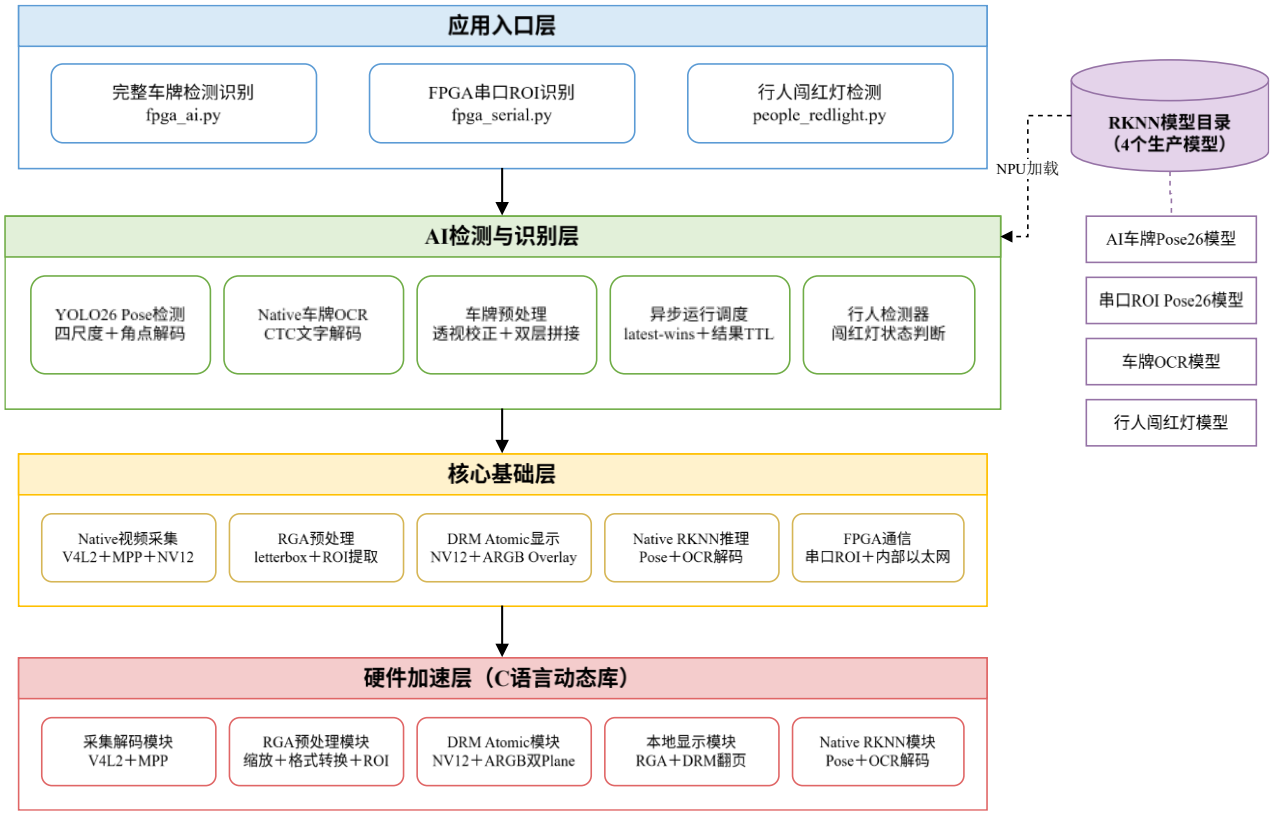


图 8 嵌入式端软件架构

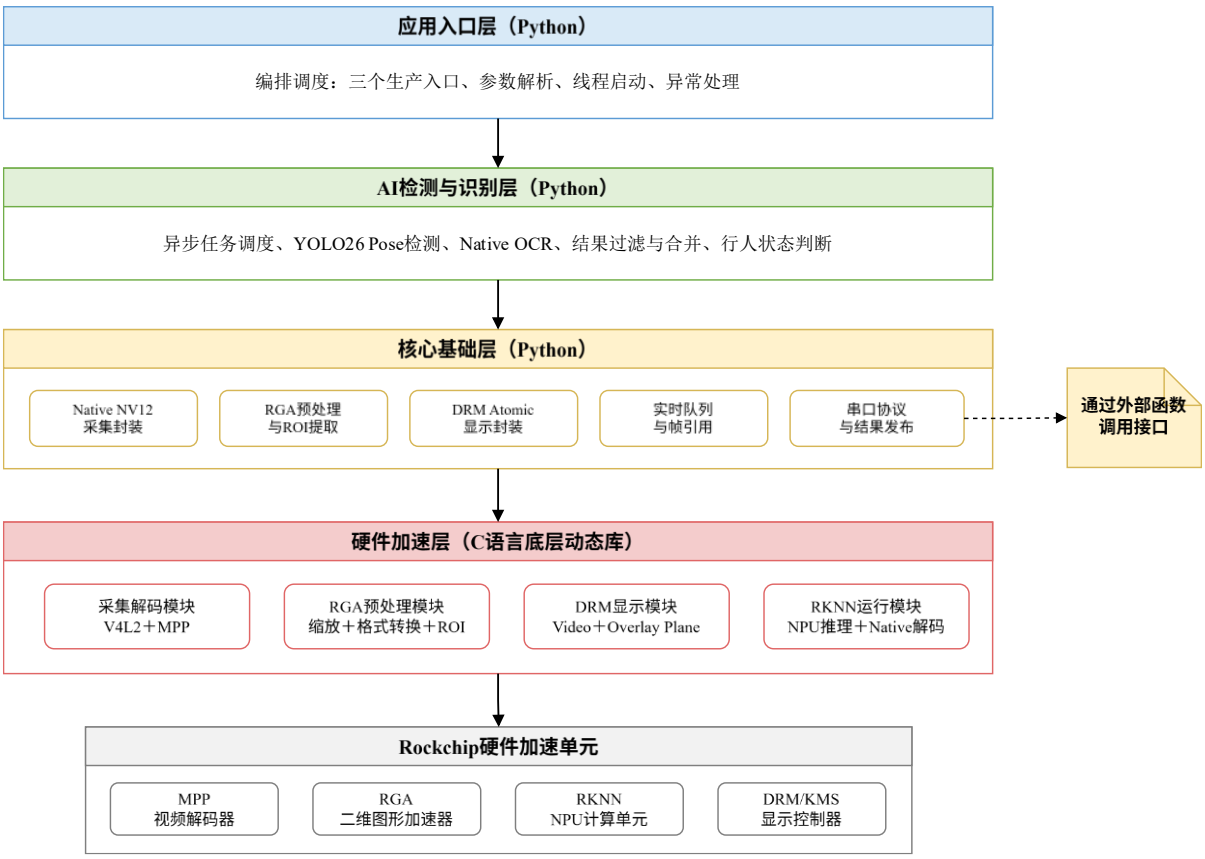


图 9 各层调用关系

进程与线程模型：系统采用单进程、多线程、事件驱动的异步流水线架构。各线程共享 Native NV12 帧对象，通过引用计数传递 DMA-BUF，减少大尺寸图像复制。任务队列采用单槽 latest-wins 机制：当处理速度低于视频输入速度时，待处理旧任务会被最新任务覆盖，从而保证系统优先处理最新画面。

以 fpga_ai.py 为例，主线程负责整体调度，主线程之外包含视频采集、车牌检测、车牌识别和画面显示四个主要工作线程。启用上位机发布功能时，系统还会启动独立的 Publisher 线程。

主线程：事件驱动调度

主线程负责初始化硬件和模型、启动工作线程、转发 Native 帧引用、合并检测与识别结果、维护显示结果 TTL 以及处理异常恢复。采集帧、检测结果、识别结果或错误到达时触发共享事件；无任务时主线程阻塞等待。

Main Thread
Input: capture events, detection results, recognition results **Output:** display tasks, detection tasks, Overlay state
1. initialize capture, display and RKNN models 2. start capture, detection, recognition and display workers

```
3.  while not stop do
4.      wait for shared wake event
5.      frame ← acquire latest Native NV12 frame
6.      dispatch retained frame references to display and detection
7.      receive latest detection and recognition results
8.      merge current results and check result TTL
9.      if Overlay content changed then
10.         submit updated Overlay data
11.     handle worker errors and hardware reconnection
12.   stop workers and release all frame references
```

线程 1：视频采集线程

视频采集线程位于底层 C 动态库中，负责 V4L2 视频采集和 MPP 硬件解码。线程通过 V4L2 MMAP 方式获取 1920×1080、60fps 的 MJPEG 码流，调用 MPP 解码器生成 NV12 格式图像，并将解码结果保存在可复用的 DMA-BUF 帧池中。Python 层接收轻量化的 NativeNv12Frame 对象，其中保存 DMA-BUF 文件描述符、图像尺寸、步长、时间戳和引用令牌。采集队列采用 latest-wins 机制，覆盖旧帧时同步释放其引用。

```
Thread 1: Native Capture Thread
Input:  HDMI capture device, MJPEG 1920×1080@60fps
Output: latest NativeNv12Frame
1. v4l2_dev ← open and configure V4L2 MJPEG device
2. mpp_dec ← initialize MPP MJPEG decoder
3. frame_pool ← allocate reusable NV12 DMA-BUF slots
4. mmap_buffers ← initialize V4L2 MMAP buffers
5. while not stop do
6.      jpeg_buf ← VIDIOC_DQBUF
7.      packet ← wrap MPP packet
8.      nv12_frame ← MPP decode into DMA-BUF frame pool
9.      frame_info ← create Native frame reference
10.     publish_latest(frame_info)
11.     if old frame is overwritten then
12.         release old frame reference
13.     VIDIOC_QBUF(jpeg_buf)
```

线程 2：检测线程

检测线程接收 Native NV12 帧引用，通过 RGA 将 NV12 DMA-BUF 直接转换并缩放为 512×288 RGB NHWC 模型输入。系统设置两个持久化模型输入缓冲，分别保存当前推理输入和下一帧预处理输入。YOLO26 Pose 模型在 NPU 上完成车牌检测，Native C 代码负责读取 P2 至 P5 四个输出头，完成反量化、Sigmoid、

全局 Top-K 解码和坐标恢复。每个检测结果包含车牌边界框、检测置信度、四个角点、角点可见度和单双层类别。检测结果经过置信度及几何过滤后发送至识别线程。

Thread 2: Plate Detection Thread
Input: latest NativeNv12Frame **Output:** detection results and recognition tasks
1. initialize Pose26 RKNN model 2. allocate two persistent model input buffers 3. **while** not stop **do** 4. task ← wait for latest detection task 5. slot ← acquire free input slot 6. RGA letterbox NV12 DMA-BUF to 512×288 RGB NHWC 7. run Pose26 model on NPU 8. identify P2–P5 output heads 9. decode global Top-K candidates 10. restore bbox and landmarks to 1920×1080 coordinates 11. apply confidence and geometric filtering 12. publish latest detection results 13. retain frame for valid recognition tasks 14. release detection frame reference

线程 3：识别线程

识别线程接收检测结果及对应的 Native 帧引用。系统通过 RGA 从 NV12 DMA-BUF 中提取小尺寸 BGR 车牌区域，再使用 OpenCV 根据四个角点完成透视校正。双层车牌经过分割和重新拼接后转换为单行形式，随后使用 OpenCV 调整为 168×48 BGR。处理后的车牌图像送入 Native OCR 模型。底层 C 代码复用持久化输入和输出缓冲，完成 RKNN 推理、稳定 Softmax、21 步 CTC 去重去空白和颜色类别解码。Python 层负责将字符索引映射为实际车牌字符。

检测模型和识别模型通过统一的 NPU 执行仲裁器串行调用，避免两个 RKNN 模型同时占用 NPU 资源。

Thread 3: Recognition Thread
Input: detection results and retained NativeNv12Frame **Output:** latest plate recognition results
1. initialize Native OCR model 2. **while** not stop **do** 3. task ← wait for latest recognition task 4. for each valid plate detection do 5. roi_bgr ← RGA extract BGR ROI from NV12 DMA-BUF 6. plate ← OpenCV perspective transform

```
7.              if class is double-layer then
8.                  plate ← split and merge into one row
9.              plate ← OpenCV resize to 168×48 BGR
10.             acquire shared NPU execution lock
11.             outputs ← run Native OCR model
12.             release shared NPU execution lock
13.             indices ← stable Softmax and CTC decode
14.             plate_text ← map indices to plate characters
15.             apply recognition confidence filtering
16.             append valid recognition result
17.         release Native frame reference
18.         publish the complete recognition batch
```

线程 4：显示线程

显示线程接收 Native NV12 DMA-BUF 帧，并通过 DRM/KMS Atomic 接口输出至 HDMI。视频画面使用 NV12 Video Plane，车牌框、车牌号和调试信息使用独立的 ARGB Overlay Plane，两个图层由显示硬件完成合成。

Thread 4: Display Thread

Input: latest NativeNv12Frame, Overlay state
Output: HDMI display 1920×1080@60fps

```
1. initialize DRM/KMS video and Overlay planes
2. allocate double-buffered ARGB Overlay buffers
3. load Chinese font with FreeType
4. while not stop do
5.          frame ← wait for latest display frame
6.          retire completed Page Flip tokens
7.          if Overlay content changed then
8.              redraw plate boxes, text and debug information
9.              submit updated ARGB Overlay buffer
10.         bind NV12 DMA-BUF to video Plane
11.         perform DRM Atomic Commit and request Page Flip
12.         retain current frame until retirement token returns
13. on shutdown, retire inflight frames and close DRM resources
```

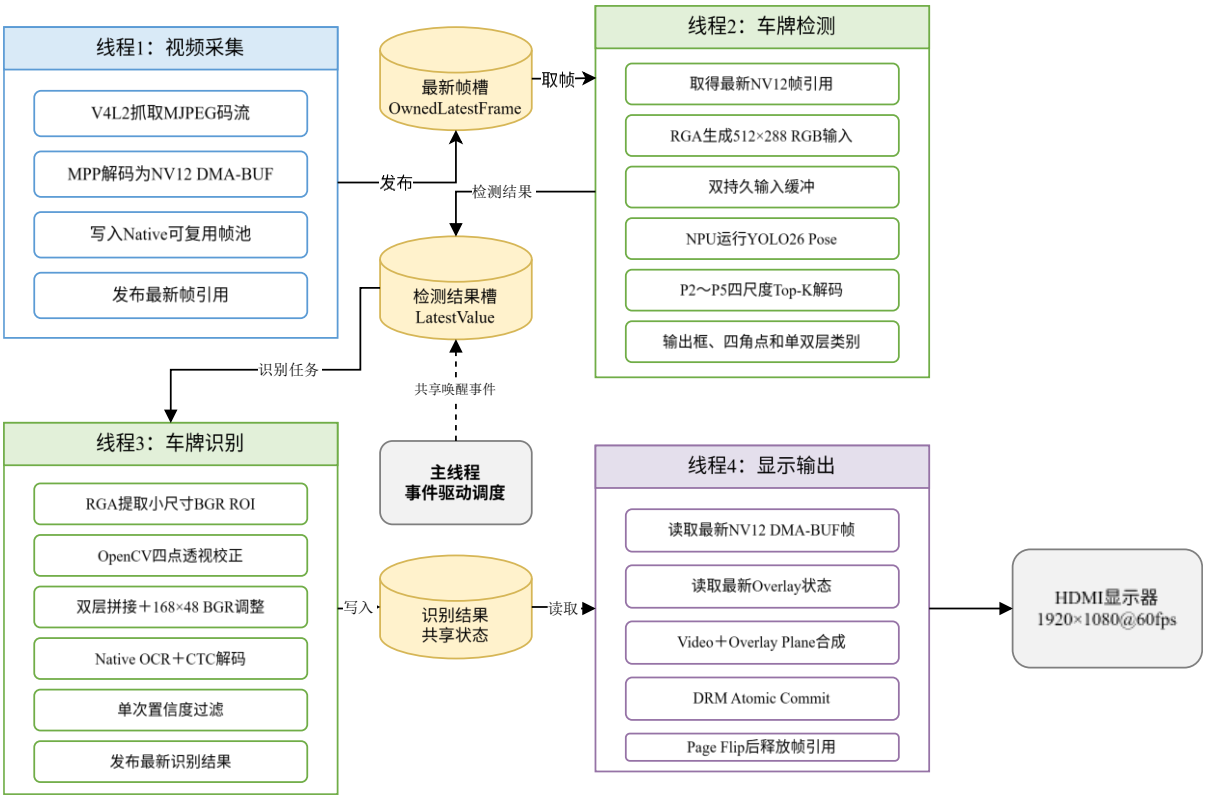


图 10 多线程异步流水线示意图

Web 管理平台架构

软件框架

```
rk3568_manager/
├── app.py                  Flask 主应用，注册路由和 WebSocket
├── config.py               全局配置
├── requirements.txt        Python 环境依赖
├── backend/                后端核心逻辑
│   ├── auth.py                 JWT 认证
│   ├── ssh_manager.py          SSH 远程指令管理器
│   └── socket_handlers.py      WebSocket 实时通信处理
├── db/                     SQLite 数据库存储目录
└── frontend/               前端 Vue 项目目录
    └── index.html          Vue3 单页面应用入口
```

Web 远程管理平台 rk3568_manager 部署在上位机，为用户提供基于浏览器的远程控制界面，支持远程启停 RK3568 上的推理功能并实时查看运行日志。

平台管理三种推理模式，每种模式对应 RK3568 上的一个入口脚本：

车牌检测和识别，执行 fpga_ai.py

串口坐标识别，执行 fpga_serial.py

行人闯红灯检测，执行 people_redlight.py

同一时刻只允许运行一种模式。

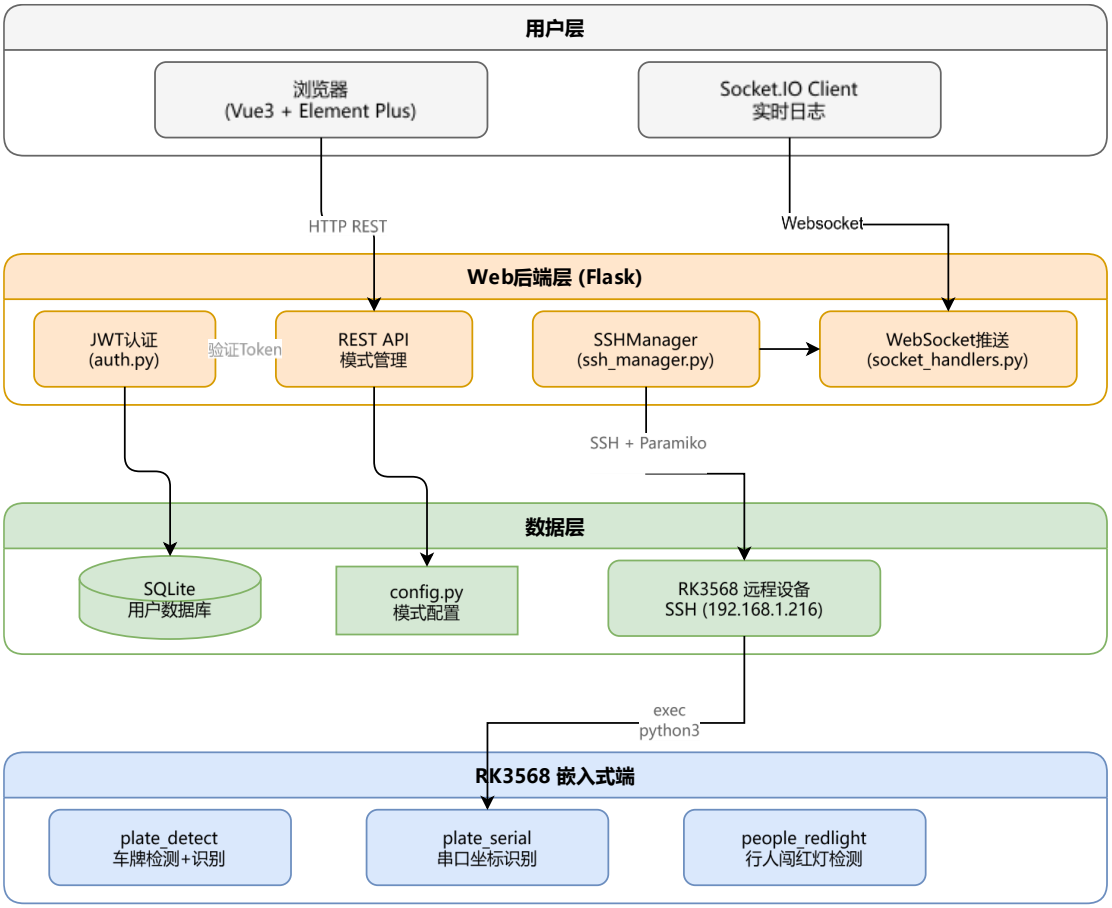


图 11 Web 端架构

后端架构：由四个核心模块组成。

app.py：Flask 主应用入口，负责注册路由蓝图、配置 JWT 认证中间件、挂载 CORS 跨域支持，并启动 Flask-SocketIO 服务。

auth.py：实现 JWT 认证蓝图，提供用户注册、登录和用户信息查询三个接口，用户数据以 bcrypt 哈希存储在 SQLite 数据库中。

ssh_manager.py：建立 SSH 连接到 RK3568 设备为每个推理模式分配 PTY 终端并执行对应的 Python 脚本，同时在后台线程持续读取输出，通过 SocketIO 推送到前端。

socket_handlers.py：注册 WebSocket 事件处理器，连接时验证 JWT Token，支持

模式状态变更通知和实时日志流推送两个事件。

前端架构：单文件 SPA，使用 Vue 3 Composition API 和 Element Plus 2.7.0 构建。包含登录页和管理控制台两个页面。

控制台顶部显示导航栏，左侧展示三个模式卡片，右侧为实时终端日志面板。



图 12 登录页展示

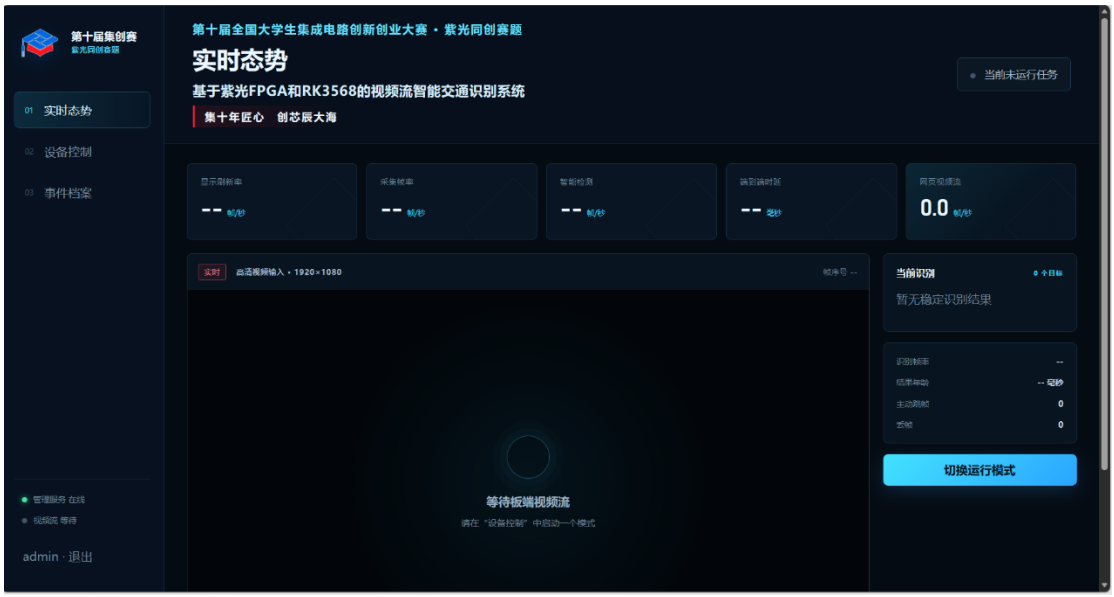


图 13 设备管理界面

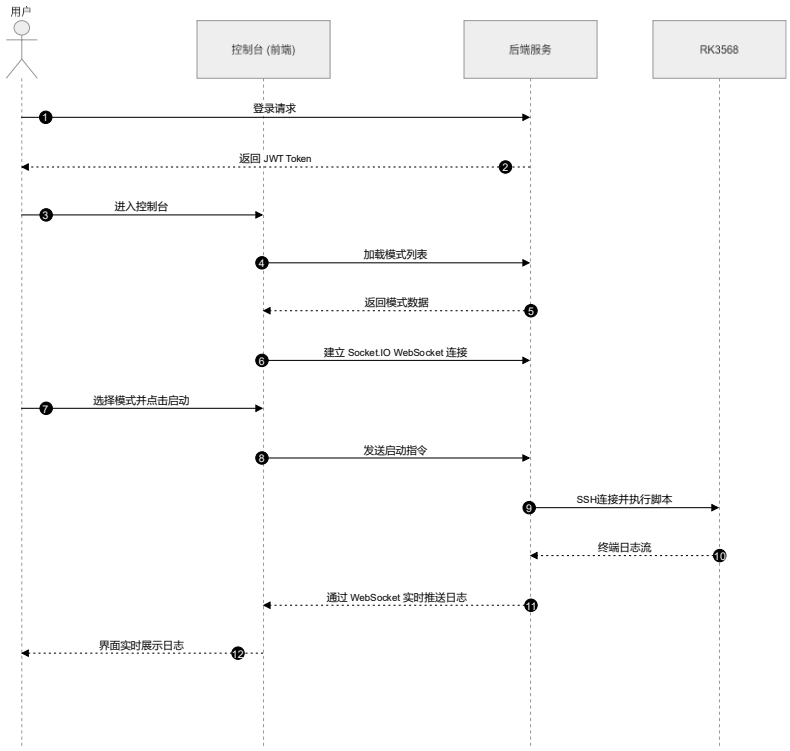


图 14 交互流程

底层核心模块设计

core/native_capture.py：HDMI 硬件采集

功能：封装底层 Native 采集库，完成 V4L2 视频采集、MPP 硬件 MJPEG 解码和 NV12 DMA-BUF 帧管理。

运行过程：NativeMppHdmiReceiver 启动后配置采集设备、分辨率、帧率和视频格式，并创建后台采集线程。底层 C 库通过 V4L2 MMAP 方式获取 1920×1080、60fps 的 MJPEG 码流，使用 MPP 解码为 NV12 格式，将图像保存在可复用的 DMA-BUF 帧池中，再向 Python 发布轻量 Native 帧对象。队列满时覆盖旧帧并释放其引用。

core/rga_preprocess.py：RGA 硬件预处理

功能：封装 librga_preprocess.so，使用 RGA 完成模型输入预处理和小尺寸车牌 ROI 提取。

运行过程：模块接收 BGR 图像或 NV12 DMA-BUF，根据模型尺寸计算缩放比例和 letterbox 填充区域，再调用 RGA 完成图像缩放、颜色格式转换和模型输入写入。处理车牌时，模块根据检测框从 NV12 帧中提取小尺寸 BGR 区域，并返回实际对齐后的 ROI 范围，供后续透视校正和坐标换算使用。

core/rknn_native.py：Native RKNN 推理封装

功能：封装 librknn_runtime.so，负责 RKNN 模型加载、输入输出张量查询、持久缓冲复用和 NPU 推理。

运行过程：程序启动时，NativeRknnModel 加载 RKNN 模型并读取输入输出张量信息，根据模型格式创建持久输入和输出缓冲。检测模型使用两个输入缓冲，RGA 在空闲缓冲中准备下一帧；NPU 在当前缓冲上执行推理并由 Native C 代码完成 Pose 结果解码。OCR 模型复用固定输入输出缓冲，完成 NPU 推理、Softmax 和 CTC 文字解码。

core/display_drm_atomic.py：DRM Atomic 零拷贝显示

功能：封装 librkdrm_atomic.so，通过 DRM/KMS Atomic 接口完成 NV12 视频和 ARGB 标注图层显示。

运行过程：显示模块初始化 Video Plane 和 Overlay Plane，Video Plane 直接绑定 MPP 输出的 NV12 DMA-BUF，Overlay Plane 绘制检测框、车牌号和调试信息。每帧通过 Atomic Commit 提交，并在 Page Flip 完成后释放已过期的视频帧引用。检测或识别结果发生变化以及 TTL 到期时更新 Overlay，视频画面按照采集速率逐帧显示。

core/display_drm_rga.py：本地视频显示

功能：负责本地 BGR 视频的 RGA 转换和 DRM 双缓冲输出。

运行过程：行人模式读取本地视频时，OpenCV 首先将压缩视频解码为 BGR 图像，RgaDrmDisplay 再调用底层 Native 库，通过 RGA 完成图像缩放和像素格式转换，并写入非活跃 DRM 缓冲区。转换完成后执行 Page Flip 切换显示缓冲，使视频帧输出到 HDMI 显示器。

core/realtime_pipeline.py：实时异步流水线

功能：提供 latest-wins 队列、结果槽、帧引用管理和通用显示线程。

运行过程：采集线程产生新帧后，将帧引用发布到 latest-wins 队列。检测、识别和显示线程按需要调用 retain()获得独立引用，任务被覆盖、处理完成或出现异常时调用 release_frame()释放引用。显示线程在 Page Flip 完成后释放扫描帧，确保 DMA-BUF 不会在显示控制器使用期间被提前复用。

core/publisher.py：上位机数据发布

功能：向上位机发送原始 MJPEG 视频、检测结果和运行状态。

运行过程：启用发布功能后，Native 采集线程按照设定频率保存最新原始 MJPEG 快照，业务线程同步更新检测结果和运行状态。Publisher 线程从 latest-wins 发送槽中取得最新数据，通过 HTTP 连接发送到上位机。

core/latency_metrics.py：时延统计

功能：统计视频采集、MPP 解码、任务排队、RGA 预处理、NPU 推理、结果调度、Overlay 更新和端到端显示时延。

运行过程：视频帧经过采集、解码、检测、识别和显示阶段时，各模块记录对应的单调时钟时间戳。latency_metrics.py 根据这些时间戳计算各阶段耗时，并生成平均值、P95 等统计结果。端到端时延以 V4L2 源帧时间戳为起点，以实际 Page Flip 完成时间为终点。

core/serial_io.py：FPGA 串口 ROI 协议

功能：通过 UART 接收 FPGA 发送的 1 至 4 个车牌矩形 ROI 坐标。

串口数据包格式为：

0xAA	count: uint8，范围 0～4	count × [x1, y1, x2, y2]: uint16 little-endian

坐标为 1920×1080 图像中的绝对像素 XYXY 坐标。

core/fpga_io.py：FPGA UDP 视频流接收

功能：独立进程接收 FPGA 通过 UDP 发送的 RGB565 视频帧，处理粘包问题，将最新帧放入共享队列。

参数	数值	说明
FPGA 地址	192.168.1.10:61440	FPGA 设备 IP 和端口
RK3568 地址	192.168.1.102:61440	本地接收地址和端口
帧头标记	b\xf0\x5a\xa5\x0f	4 字节帧头
帧分辨率	640×480	每帧像素尺寸
帧数据大小	614400 字节	RGB565 数据载荷
头部偏移	8 字节	帧头+分辨率信息

表 4 串口数据包参数信息

触发机制：进程启动后向 FPGA 发送 1 字节触发信号，FPGA 收到后开始持续发送视频帧。

粘包处理算法：由于 UDP 是无连接协议，多个数据包可能合并到达。模块使用

字节缓冲区，通过搜索帧头标记定位帧边界，当缓冲区中数据长度足够时截取完整帧数据。缓冲区过大时自动清空，防止内存泄漏。

RGB565 解码：将大端 RGB565 字节流通过 NumPy 视图重组后调用 OpenCV 转换为 BGR 图像。

车牌检测识别模块

plate_det/ai_runtime.py：RK 独立运行

功能：RK3568 独立车牌检测识别异步流水线，实现接收 NV12 帧、RGA 双缓冲预处理、YOLO26-Pose 检测、调度 OCR、结果合并、HDMI Overlay 刷新，处理设备断连与 RKNN 异常。

运行过程：运行时从 Native 采集模块取得最新 NV12 帧，将帧引用分别发送给检测线程和显示线程。检测线程通过 RGA 生成模型输入并运行 YOLO26 Pose 模型，识别线程根据检测框和四角点提取车牌、完成透视校正并调用 OCR 模型。主线程合并检测与识别结果，内容变化时更新 Overlay，显示线程持续输出 HDMI 画面。

plate_det/serial_runtime.py：串口模式运行

功能：实现 FPGA 串口 ROI 车牌检测与识别流程。

运行过程：运行时接收 FPGA 通过串口发送的 1 至 4 个 XYXY 区域，将这些区域与最新 NV12 帧绑定，并通过 RGA 组成四槽模型输入。YOLO26 Pose 模型进一步检测车牌四角点和单双层类别，随后完成车牌透视校正、双层拼接和 OCR 识别，最后统一更新 HDMI 及上位机结果。收到空包或非法坐标时立即清空当前结果。

plate_det/detection_filter.py：检测候选过滤

功能：根据置信度和连续帧状态过滤车牌检测候选。

运行过程：模块首先检查候选框、四角点和置信度是否有效。高置信度候选直接进入识别流程，低置信度候选则与前几帧结果进行位置匹配，连续出现达到设定次数后才予以确认，从而减少偶发虚检对 OCR 和显示结果的影响。

plate_det/plate_pipeline.py：车牌预处理流水线

功能：完成车牌 ROI 提取、透视校正、双层拼接和 OCR 输入准备。

运行过程：模块根据检测框从 NV12 帧中提取小尺寸 BGR 区域，再将四角点转

换为 ROI 局部坐标并执行透视校正。对于双层车牌，系统将上下两部分重新拼接为单行，最后使用 OpenCV 调整为 168×48 BGR，作为车牌 OCR 模型的输入。

plate_det/plate_recognizer.py：车牌 OCR 识别

功能：执行车牌文字识别和内部颜色分类。

运行过程：模块将处理后的 168×48 BGR 车牌图像送入 RKNN OCR 模型，底层完成 NPU 推理、Softmax 和 21 步 CTC 解码，去除重复字符及空白符。Python 根据字符索引生成最终车牌号码。

plate_det/plate_recognition.py：字符表与双层拼接

功能：保存车牌字符表、颜色类别，并提供双层车牌拼接工具。

运行过程：OCR 解码时，模块根据模型输出索引查询字符表，将索引转换为省份简称、字母、数字或特殊车牌字符。处理双层车牌时，模块分别截取车牌的上下区域，调整为合适尺寸后水平拼接，生成符合 OCR 模型输入形式的单行车牌图像。

plate_det/result_utils.py：结果处理工具

功能：提供检测结果完整性判断和边界框交并比计算。

运行过程：模块在检测过滤和结果合并过程中检查车牌号、边界框、四角点等字段是否完整，并计算不同检测框之间的 IoU，用于判断相邻帧中的候选是否属于同一车牌。

行人检测模块

people_det/people_detector.py：行人闯红灯检测器

功能：封装行人闯红灯 RKNN 模型的预处理、NPU 推理、后处理和标注数据生成。

运行过程：模块先将输入图像等比例缩放至模型尺寸并转换为 RGB 格式，再调用 RKNN 模型执行 NPU 推理。推理结果经过置信度过滤、NMS 和坐标恢复后生成检测目标，其中普通行人使用绿色标注，闯红灯目标使用红色标注，并同时统计各类目标数量。

people_det/runtime.py：行人检测运行

功能：实现行人闯红灯检测的异步采集、NPU 推理和显示流程。

运行过程：HDMI 模式下，采集线程持续生成 NV12 帧，显示线程逐帧输出最新

画面，推理线程使用 latest-wins 机制处理最新检测任务，并在结果变化时更新 Overlay。本地视频模式则由 OpenCV 顺序读取视频帧，完成检测后通过 DRM-RGA 或窗口显示处理结果。

people_det/app.py：行人模式入口配置

功能：负责行人模式的命令行参数、模型路径、类别名称、显示后端和输入方式选择。

运行过程：程序启动时解析命令行参数，加载类别文件和行人闯红灯模型，并根据是否提供视频路径选择 HDMI 实时模式或本地视频模式。完成检测器和显示设备初始化后，模块调用对应运行时；程序退出时按顺序释放模型、显示和采集资源。

性能与优化说明

全国总决赛方案沿用区域决赛的 MPP 硬件解码、RGA 硬件预处理、DRM 显示和多线程异步流水线，重点对视频帧传递方式和车牌检测模型进行了升级。视频链路采用 NV12 DMA-BUF 零拷贝机制，车牌检测模型升级为 YOLO26 Pose，在保持 1920×1080、60fps 输入输出能力的同时，降低了图像复制和 CPU 内存访问开销，并提升了小尺寸、倾斜车牌的检测能力。

指标	区域决赛	总决赛	提升幅度	优化来源
小车牌召回率	72.4%	89.1%	提升 16.7%	YOLO26 Pose 增加 P2 高分辨率检测头，增强小目标特征提取
车牌检测 NPU 耗时	30ms	18ms	降低 33.3%	YOLO26 轻量化网络、INT8 量化及较小模型输入
进程 CPU 占用率	75%	33.40%	降低 55.5%	NV12 DMA-BUF 零拷贝，避免 1080p BGR 整帧转换与复制

表 5 模型性能优化数据

零拷贝视频链路

全国总决赛采用 NV12 DMA-BUF 零拷贝视频链路：V4L2 采集 MJPEG 码流，MPP 解码直接写入可复用 NV12 DMA-BUF 帧池全程无 1080P BGR 大尺寸中间帧，大幅减少 CPU 内存访问与拷贝开销。

YOLO26 Pose 模型升级

RK 独立车牌模式使用 512×288 输入的 YOLO26 Pose 模型。模型具有 P2、P3、P4 和 P5 四个输出尺度，其中 P2 高分辨率特征图增强了小尺寸车牌检测能力。模型直接输出车牌边界框、四个角点、角点可见度及单双层类别。

模型输出由 Native C 代码完成读取、反量化、稳定 Sigmoid 和全局 Top-K 解码。板上实测纯 NPU 耗时约 18.5～18.6ms，包括 RGA 预处理、输出读取和解码在内的单次检测总计算耗时约 24.5～25.8ms。

串口 ROI 检测流程

串口模式接收 FPGA 发送的 1 至 4 个 1080p XYXY 车牌候选区域。RGA 将这些区域组成 160×320 RGB 四槽画布，再送入串口专用 YOLO26 Pose 模型，进一步检测车牌四角点和单双层类别。

ROI Pose 模型纯 NPU 推理 Mean 约 7.2ms、P95 约 7.6ms。检测完成后，系统根据四角点执行 OpenCV 透视校正，双层车牌经过拼接后调整为 168×48 BGR，再送入 Native OCR 模型，OCR 纯 NPU 耗时约 1.3ms。

与纯 RK 全帧检测模式相比，串口模式利用 FPGA 提供的候选区域缩小检测范围，检测 NPU 耗时由约 18.6ms 降至约 7.2ms，降低约 61.3%。检测与 OCR 的核心 NPU 处理时间由约 19.9ms 降至约 8.5ms，降低约 57.3%，因此车牌结果的端到端响应时延也相应缩短。

1.2.3. FPGA 车牌检测模型的训练与底层优化

1. 训练过程

本模型首先读取原始 RGB 彩色车牌图像，将图像按 FPGA 固定接口调整至 320×180，并在上下各填充 6 行，形成 3×192×320 的 RGB 输入张量。模型采用轻量 YOLOv5 P2/P3 检测结构，由 7 个 Conv+BN+ReLU 主干层和双尺度检测头组成，首层卷积输入通道为 3，适配 RGB888 数据。训练阶段使用 RGB FP32 模型从零开始训练 100 轮，输入尺寸固定为 320×192，采用 batch size 64、SGD 优化器、动量 0.937、权重衰减 0.0005、学习率预热和线性衰减，并使用 Mosaic、缩放、平移和水平翻转等数据增强。损失函数采用 YOLOv5 的 CIoU 边界框回归损失、目标置信度 BCE 损失和分类损失，从而得到 FP32 最佳模型。在部署验证阶段，首先使用验证集对 FP32 模型进行激活范围校准，并采用

99.99% 分位数确定各主干层的量化尺度。随后融合 Conv 与 BN，将卷积权重按输出通道对称量化为 INT8，将 Bias 和卷积累加结果表示为 INT32，将主干 ReLU 输出量化为 UINT8，并将检测头输出定点化为 INT16 Q6.10，生成 FPGA 使用的 .mem 参数文件和 model.json 模型清单。完整验证集上的 PTQ INT8 结果为 Precision=0.9386、Recall=0.9334、mAP50=0.9666、mAP50-95=0.6907，相比 FP32 基线的 mAP50 和 mAP50-95 分别下降约 0.0014 和 0.0078，满足当前 FPGA 部署候选的精度要求。

此外，基于同一 FP32 模型进行了 25 轮 FPGA 对齐 QAT 试验，用于比较量化感知训练效果。QAT 训练阶段采用较低学习率，并关闭 Mosaic、缩放和平移增强，以保持固定输入几何。QAT 整数评估结果为 mAP50=0.9605、mAP50-95=0.6700，低于 PTQ 结果，因此最终 FPGA 参数采用 PTQ 导出结果，而不是 QAT 权重。导出包同时提供逐层 INT32 累加值、UINT8 激活值和 INT16 检测输出向量，用于后续 RTL/Verilator bit-exact 对拍验证。

2. 模型架构

模型输入为 RGB888 彩色图像，经过预处理后形成 3×192×320 的输入张量。模型由 7 个连续的 Conv+BN+ReLU 层和 2 个 1×1 Detect 卷积组成，不包含 CSP、残差结构、SPPF、PAN、上采样或特征拼接，是面向 FPGA 部署的 YOLO 检测器。主干网络逐步提取 P2 和 P3 两个尺度的特征，检测类别数为 1，每个尺度使用 3 个 anchor。

层	输入→输出通道	卷积	步长	输出尺寸
0	3→16	3×3	2	16×96×160
1	16→24	3×3	2	24×48×80
2	24→32	3×3	1	32×48×80
3	32→32	3×3	1	32×48×80，P2
4	32→48	3×3	2	48×24×40
5	48→64	3×3	1	64×24×40
6	64→64	3×3	1	64×24×40，P3
Detect 0	32→18	1×1	1	3×48×80×6
Detect 1	64→18	1×1	1	3×24×40×6

表 6 模型架构

3. 训练过程

阶段一：RGB 输入与数据处理

模型使用原始 RGB 彩色车牌图像，不再转换为单通道灰度图。图像经过固定几何预处理后调整到 320×180，并在上下各补 6 行像素，形成 320×192 的输入尺寸。

阶段二：FP32 训练

模型采用随机初始化权重进行 100 个 epoch 的 RGB FP32 从零训练，输入尺寸固定为 320×192，训练批量为 64，在单张 GPU 上运行并启用 8 个数据加载进程。优化器采用 SGD，动量为 0.937，权重衰减为 0.0005，初始学习率为 0.01，经过 3 个 epoch 的学习率预热后采用线性衰减策略。

训练阶段使用 Mosaic、平移、缩放和水平翻转等数据增强，其中 Mosaic 概率为 1.0，平移幅度为 0.1，缩放幅度为 0.5，水平翻转概率为 0.5。模型采用 YOLOv5 的 CIoU 边界框回归损失、目标置信度 BCE 损失和分类损失。由于数据集只有一个车牌类别，分类分支的损失为 0。

训练过程中使用自动混合精度 AMP、模型参数 EMA 和每个 epoch 的验证与最优权重保存机制。当前重新评估得到的 FP32 基线结果为：

Precision = 0.941247
Recall = 0.934769
mAP50 = 0.967936
mAP50-95 = 0.698507

阶段三：INT8 量化验证

最终采用 PTQ，首先使用 FP32 最佳权重在验证集上进行激活校准，从 512 张验证图像中采集 7 个主干层的激活分布，并使用 99.99% 分位值确定各层 UINT8 激活尺度。随后融合 Conv 与 BN，对融合后的卷积权重执行逐输出通道对称 INT8 量化，并依据输入激活尺度和权重尺度将 Bias 量化为 INT32。部署计算格式为：

输入：RGB UINT8

主干激活：UINT8

卷积权重：逐输出通道 INT8

Bias 和卷积累加：INT32

主干输出：UINT8，带 ReLU 和饱和处理

Detect 输出：INT16 Q6.10

PTQ INT8 完整验证集结果为：

Precision = 0.938628

Recall = 0.933393

mAP50 = 0.966575

mAP50-95 = 0.690704

PTQ INT8 量化后的精度下降较小，结果满足进入 FPGA RTL 准备阶段的条件。

阶段四：FPGA 参数导出与 RTL 验证准备

完成 PTQ 校准后，导出 FPGA 所需的模型结构和定点参数，包括：

- 7 个主干卷积层；
- 2 个 P2/P3 Detect 卷积层；
- INT8 权重；
- INT32 Bias；
- INT32 重定标乘数；
- 移位参数；
- 网络尺寸、stride、anchor 和 Detect 配置。

同时生成 RTL/Verilator 位精确验证向量。检测头输出再按照 YOLO 解码公式进行边界框解码，并执行置信度筛选、Top-K 和 NMS。

4. 训练超参数设置

在上述模型和分阶段训练架构基础上，各阶段训练超参数设置如下。

参数	FP32 训练	PTQ 校准/导出	FPGA 对齐 QAT 对比
初始化权重	随机初始化	FP32 最佳权重	第一代 QAT 最佳权重
Epoch	100	/	25
Batch size	64	校准样本 512 张	64
输入尺寸	3×192×320	3×192×320	3×192×320
优化器	SGD+Nesterov	/	SGD+Nesterov
初始学习率	0.01	/	0.001
最终目标学习率	0.0001	/	0.0001
学习率策略	线性衰减	/	线性衰减
动量	0.937	/	0.937
权重衰减	0.0005	/	0.0005
Warmup	3 epochs	/	1 epoch
Warmup 动量	0.8	/	0.8
Warmup 偏置学习率	0.1	/	0.01

表 7 模型训练超参数

5.训练与量化效果

下表采用当前重新评估得到的完整验证集结果：

阶段	验证图像数	Precision	Recall	mAP50	mAP50-95
RGB FP32 基线	4690	0.941247	0.934769	0.967936	0.698507
PTQ INT8	4690	0.938628	0.933393	0.966575	0.690704
FPGA 对齐 QAT	4690	0.936068	0.923698	0.960497	0.670002

表 8 完整验证集结果

PTQ 相对于 FP32 的变化为：

模型	ΔPrecision	ΔRecall	ΔmAP50	ΔmAP50-95
PTQ INT8 − FP32	-0.002620	-0.001375	-0.001361	-0.007803
QAT INT8 − FP32	-0.005180	-0.011070	-0.007439	-0.028505

表 9 PTQ 与 QAT 训练结果对比

PTQ INT8 的精度下降小于 QAT INT8；最终 FPGA 部署采用 PTQ INT8 参数。

1.2.4 FPGA 端算法模块介绍

1. 算法整体架构总览

FPGA 内部完成 HDMI 视频采样、图像预处理、CNN 推理、检测框解码、候选框筛选、Top-K 排序、NMS、视频叠加及检测结果接口输出。

系统接收 1920×1080@60 Hz HDMI RGB888 视频，输入数据经 pixclk_in 采样后分为两条并行路径。第一条为显示路径，采样后的 RGB888 像素直接转换为 RGB565，并通过 yolo_box_overlay 进行实时矩形框叠加，随后恢复为 RGB888 输出至 HDMI 显示。第二条为 YOLO 检测路径，原始 R、G、B 三个 8 位通道送入 yolo_preprocess_6x6，经过 6×6 区域平均下采样和上下 Letterbox 填充，生成 320×192×3 的 RGB 输入图像，并写入 yolo_input_frame_buffer 的片上双 Bank 缓存。yolo_frame_mailbox 负责完成帧号和 Bank 所有权传递，CNN 从已完成的 Bank 读取输入数据。yolo_cnn_engine 依次执行 7 层骨干卷积和 2 个检测头共 9 个网络操作，输出两个尺度的 INT16 Q6.10 检测 logit；随后由 yolo_postprocess 完成检测头通道组装、Sigmoid、边界框解码、置信度阈值筛选、Top-32 排序和 NMS，最终输出最多 4 个车牌候选框的轴对齐矩形坐标和置信度。检测结果一方面送入 yolo_box_overlay，在后续 HDMI 视频流中叠加红色矩形框；另一方面可通过外部 UART 接口发送至 RK3568，由 RK3568 进一步完成车牌精确定位、四角点检测和字符识别。

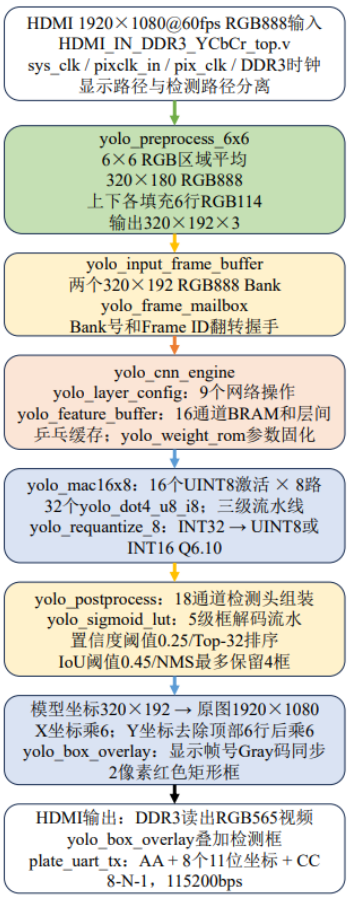


图 15 FPGA 端车牌检测流程图

2. 图像预处理与输入控制

2.1 6 倍降采样

yolo_preprocess_6x6

模块功能：是 YOLO 检测通路前端的固定比例 RGB888 预处理单元。它接收 1920×1080（1080p）的 24-bit RGB 视频流，通过 6×6 块内均值下采样（遵循 OpenCV INTER_AREA 的 round-to-nearest-even 舍入）将分辨率压缩至 320×180，再在上下各填充 6 行灰度值 0x727272，最终输出 320×192 的 HWC 行优先像素流。输出写接口与下游特征读取器无缝衔接，R/G/B 分别作为通道 0/1/2。

实现逻辑：该模块实现 6×6 下采样预处理：每行内以 horizontal_phase 计数，累加 6 个像素的 R/G/B 至 11 位寄存器，水平块完成时触发垂直累加；垂直方向以 vertical_phase 记录行数，利用 vertical_r/g/b 数组保存每列 6 行内的累加和（13 位），当完成第 6 行时启动除法状态机，将三通道累加和依次送入单端口 ROM divide_36_rom（地址 0~9180，存储向偶数舍入的均值），依次读取 R/G/B

结果并组合成写数据。帧起始时先通过 padding_active 在非视频期输出上下各 6 行填充像素（0x727272），有效内容从 TOP_PIXELS 地址开始连续写入，每完成一个块地址递增；帧同步下降沿复位所有计数并开始填充，line_seen 标记行有效，帧结束检查 active_y 是否为 1080，否则报 geometry_error。整体流水线每 6 个时钟完成一个块，除法占用 3 周期，满足实时处理需求。

关键代码：

```
wire [13:0] block_sum_r = {1'b0, vertical_r[output_x]} +
                          {3'b000, horizontal_r} +
                          {6'b000000, r_in};
if (vertical_phase == SCALE - 1) begin
    divide_rom_addr      <= block_sum_r;
    pending_sum_g        <= block_sum_g;
    pending_sum_b        <= block_sum_b;
    pending_write_addr <= content_write_addr;
    divide_state         <= DIVIDE_ISSUE_G;
end
```

三个通道查表完成后组合为 RGB888 像素：

```verilog
DIVIDE_CAPTURE_B: begin
    write_en    <= 1'b1;
    write_addr <= pending_write_addr;
    write_rgb   <= {divided_r, divided_g, divide_rom_data};
    if (pending_final) begin
        frame_done <= 1'b1;
        frame_id    <= frame_id + 1'b1;
    end
end
```

2.2 帧缓存写入

yolo_input_frame_buffer.v

模块功能：降采样后的有效内容为 320x180，而网络输入为 320x192。因此预处理模块在垂直方向上方和下方各写入 6 行 RGB 114 像素。完整输入帧包含 61440 个 24 bit 像素，由 yolo_input_frame_buffer.v 的两个 Bank 保存。

实现逻辑：输入缓存包含两个深度为 61440 的 RGB888 Bank。yolo_frame_mailbox 保存当前写 Bank；一帧完成且上一推理任务已经应答时，发布已完成 Bank 和 16 bit 帧号，然后翻转 source_write_bank。推理侧检测到发布 toggle 变化后给出 inference_job_valid、读取 Bank 和帧号。

关键代码：

```
if (padding_active && !de_in) begin
      write_en    <= 1'b1;
      write_rgb <= 24'h727272;
      if (padding_index < TOP_PIXELS)
            write_addr <= padding_index;
      else
            write_addr <= BOTTOM_BASE + padding_index - TOP_PIXELS;
end
reg [23:0] bank_0 [0:DEPTH-1];
reg [23:0] bank_1 [0:DEPTH-1];

always @(posedge write_clk) begin
      if (write_en) begin
            if (write_bank)
                  bank_1[write_addr] <= write_rgb;
            else
                  bank_0[write_addr] <= write_rgb;
      end
end
if (source_frame_done) begin
      if (acknowledge_sync_2 == publish_toggle) begin
            published_bank         <= source_write_bank;
            published_frame_id <= source_frame_id;
            publish_toggle         <= ~publish_toggle;
            source_write_bank    <= ~source_write_bank;
      end else begin
            source_overrun <= 1'b1;
      end
end
```

3. 网络结构定义与推理调度

3.1 九层网络结构

网络结构由 yolo_layer_config.v 以组合逻辑方式固定描述，共包含 7 层骨干卷积和 2 个 Detect 头。网络输入是 3 通道 RGB888，具体结构下表所示：

层号	输入尺寸	输出尺寸	卷积参数	说明
0	192×320×3	96×160×16	3×3, s=2, p=1	首层卷积，1→16
1	96×160×16	48×80×24	3×3, s=2, p=1	特征下采样，16→24
2	48×80×24	48×80×32	3×3, s=1, p=1	特征提取，24→32
3	48×80×32	48×80×32	3×3, s=1, p=1	特征提取，32→32

4	48×80×32	48×80×18	1×1, s=1, p=0	检测头 0（小尺度，3 anchors×6）
5	48×80×32	24×40×48	3×3, s=2, p=1	下采样，32→48
6	24×40×48	24×40×64	3×3, s=1, p=1	特征提取，48→64
7	24×40×64	24×40×64	3×3, s=1, p=1	特征提取，64→64
8	24×40×64	24×40×18	1×1, s=1, p=0	检测头 1（大尺度，3 anchors×6）

表 10 FPGA 端 9 层网络结构

两个检测头均输出 18 个有符号 INT16 Q6.10 通道。每个网格位置包含 3 个 Anchor，每个 Anchor 包含 tx、ty、tw、th、objectness、class_score 六个参数。

关键代码：

```
// Execution order: BB0, BB1, BB2, BB3, Detect0, BB4, BB5, BB6, Detect1.
case (operation)
    4'd0: begin end
    4'd4: begin
        input_width = 9'd80; input_height = 8'd48;
        output_width = 9'd80; output_height = 8'd48;
        input_channels = 7'd32; output_channels = 7'd18;
        kernel_width = 2'd1; kernel_height = 2'd1;
        detect_mode = 1'b1; detect_head = 1'b0;
    end
    4'd8: begin
        input_width = 9'd40; input_height = 8'd24;
        output_width = 9'd40; output_height = 8'd24;
        input_channels = 7'd64; output_channels = 7'd18;
        kernel_width = 2'd1; kernel_height = 2'd1;
        detect_mode = 1'b1; detect_head = 1'b1;
    end
endcase
```

3.2 配置描述与统一调度器

网络参数由 yolo_layer_config.v 输出，层间调度、卷积循环、地址生成和流水排空统一在 yolo_cnn_engine.v 中实现。

实现逻辑：CNN 引擎状态机 STATE_IDLE、STATE_RUN、STATE_DRAIN 三个状态。接收 start 后，引擎锁存输入 Bank 和帧号，并从操作 0 开始。STATE_RUN 中使用 output_y、output_x、output_tile、kernel_y、kernel_x、input_tile 组成嵌套循环，每拍发出一个激活值、权重和参数读取请求。当前操作的最后一个请求发出后进入 STATE_DRAIN，等待 MAC 和重量化流水线中的最后结果到达。若当

前操作不是 8，则操作号加 1 并重新进入运行状态；操作 8 排空后拉高 done 一个时钟周期。中间特征由 yolo_feature_buffer 保存。该模块由 16 个 8 bit Bank 组成，每次读取 128 bit，即 16 个相邻通道；每次写入 64 bit，即 8 个输出通道。每个 Bank 内部具有两个特征缓冲区，由层配置中的 source_buffer 和 destination_buffer 选择，实现层间乒乓存储。

关键代码：

```
localparam STATE_IDLE  = 2'd0;
localparam STATE_RUN   = 2'd1;
localparam STATE_DRAIN = 2'd2;
if (state == STATE_IDLE && start) begin
    state                <= STATE_RUN;
    operation            <= 4'd0;
    active_frame_id      <= start_frame_id;
    active_input_bank <= input_bank;
end
if (requant_output_valid && meta_valid_9 && meta_final_9) begin
    if (operation == 4'd8) begin
        state <= STATE_IDLE;
        done  <= 1'b1;
    end else begin
        state     <= STATE_RUN;
        operation <= operation + 1'b1;
    end
end
```

4. 卷积计算与定点量化

4.1 CNN 运行引擎与地址生成

yolo_cnn_engine.v

模块功能：负责输入/特征读取、卷积窗口遍历、权重参数寻址、MAC 调用、重量化结果写回和 Detect 流输出。权重、偏置、乘法因子和移位量由 yolo_weight_rom.v 从随 bitstream 初始化的 .mem 文件读取，不依赖 DDR 运行时加载。

实现逻辑：

输入坐标按以下关系计算：

```
source_x = output_x * stride + kernel_x - padding

source_y = output_y * stride + kernel_y - padding
```

坐标越界时 source_padding 为高，激活值直接置零。操作 0 从输入 RGB 帧缓存读取一个 24 bit 像素，并将 R、G、B 按通道 0、1、2 放入 128 bit 激活向量的低 24 位；后续层从 128 bit 特征缓冲读取 16 通道 Tile。输出通道按 8 通道一个 Tile 处理。request_first 表示当前输出点的第一次卷积累加，request_last 表示最后一个卷积核位置和输入 Tile。最后一次请求产生的元数据经过与计算链路匹配的寄存器延迟后，用于决定将结果写回特征缓存还是输出为 Detect logits。

关键代码：

```
wire signed [11:0] source_x = scaled_output_x + kernel_x - cfg_padding;
wire signed [10:0] source_y = scaled_output_y + kernel_y - cfg_padding;

wire source_padding = source_x[11] || source_y[10] ||
                      (source_x >= $signed({1'b0, cfg_input_width})) ||
                      (source_y >= $signed({1'b0, cfg_input_height}));
wire [127:0] selected_activation = request_padding_d ? 128'd0 :
    (request_source_is_input_d ?
        {104'd0, input_read_rgb[7:0], input_read_rgb[15:8],
input_read_rgb[23:16]} :
        feature_read_data);
assign feature_write_en = requant_output_valid && meta_valid_9
&& !meta_detect_9;
assign detect_valid        = requant_output_valid && meta_valid_9
&&   meta_detect_9;
assign detect_channel_base      = meta_tile_9 << 3;
assign detect_logits_q6_10      = requant_detect_i16;
```

4.2 八输出通道并行 MAC 阵列

yolo_mac16x8

模块功能：每次接收 16 个 UINT8 激活值，并行计算 8 个输出通道。每个输出通道配置 4 个 yolo_dot4_u8_i8，因此阵列共包含 32 个 dot4 单元；每个 dot4 单元计算 4 个 UINT8 乘 INT8 乘积及其和。一次请求在算法上并行完成 128 次 8 bit 乘法，并产生 8 路部分和。

实现逻辑：硬件综合路径中，每个 dot4 使用两个 GTP_APM_E2，以双 12x9 SIMD 模式计算 4 个乘积，再由逻辑加法器得到 20 bit 有符号部分和。8 个输出通道各自将 4 个 dot4 结果先两两相加，再合成为通道部分和。MAC 阵列使用 input_first 和 input_last 标记一次完整卷积累加的边界：首个输入 Tile 将 INT32 偏置加入部分和，中间 Tile 继续累加，最后一个 Tile 把 8 路 INT32 结果写到 `accum_i32`

并产生 output_valid。

关键代码：

```
for (output_lane = 0; output_lane < 8; output_lane = output_lane + 1) begin
    for (group = 0; group < 4; group = group + 1) begin
        yolo_dot4_u8_i8 dot4 (
            .activation_u8(activation_lane_q[output_lane][group*32 +: 32]),
            .weight_i8      (weight_lane_q[output_lane][group*32 +: 32]),
            .sum_i20         (dot4_results[(output_lane*4+group)*20 +: 20])
        );
    end
end
wire signed [19:0] behavior_sum =
    (activation_0 * weight_0) + (activation_1 * weight_1) +
    (activation_2 * weight_2) + (activation_3 * weight_3);
if (first_d)
    accumulator[lane] <= $signed(bias_d[lane*32 +: 32]) +
                                {{12{partial_registered[lane][19]}},
partial_registered[lane]};
else
    accumulator[lane] <= accumulator[lane] +
                                {{12{partial_registered[lane][19]}},
partial_registered[lane]};
```

4.3 八通道重量化流水线

yolo_requantize_8.v

模块功能：将 8 路 INT32 累加结果转换为骨干网络的 UINT8 特征或检测头的 INT16 Q6.10 logit。偏置已在 yolo_mac16x8 中加入，因此本模块的核心公式为：

scaled = (accum * multiplier + 2^(shift-1)) >>> shift

实现逻辑：模块使用 4 条共享 32x32 有符号乘法路径，在相邻两个时钟分别处理通道 0~3 和通道 4~7。每个 32x32 乘法被拆成 low*low、low*high、high*low 和 high*high 四个 16 bit 部分积，再按位权合并为 64 bit 乘积。两组通道使用 slot 标签保存对应的 detect_mode、lane 有效位和 shift 参数。骨干网络输出执行 ReLU 及 UINT8 饱和，范围为 0~255；Detect 输出饱和到有符号 INT16 范围-32768~32767。

关键代码：

```
wire signed [63:0] product_value = product_ll_wide +
                                   (product_lh_wide <<< 16) +
                                   (product_hl_wide <<< 16) +
                                   (product_hh_wide <<< 32);
function signed [63:0] round_shift;
```

```
    input signed [63:0] product_value;
    input [5:0] shift_value;
    reg signed [63:0] rounding;
    begin
        if (shift_value == 0)
            round_shift = product_value;
        else begin
            rounding      = 64'sd1 <<< (shift_value - 1'b1);
            round_shift = (product_value + rounding) >>> shift_value;
        end
    end
endfunction
if (!saved_detect_mode[lower_result_slot]) begin
    if ($signed(shared_shifted[lane*64 +: 64]) < 0)
        backbone_u8[lane*8 +: 8] <= 8'd0;
    else if ($signed(shared_shifted[lane*64 +: 64]) > 64'sd255)
        backbone_u8[lane*8 +: 8] <= 8'hff;
end
```

5. 检测输出装配与后处理

5.1 检测头 18 通道装配

yolo_postprocess.v

模块功能：18 个 Detect 通道直接在 yolo_postprocess.v 内部装配。CNN 每拍最多输出 8 个 logit，并通过 detect_channel_base 指示当前通道组，依次发送通道 0~7、8~15 和 16~17。

每个网格点的通道排列为：

通道 0~5 : Anchor0 的 tx、ty、tw、th、obj、cls

通道 6~11 : Anchor1 的 tx、ty、tw、th、obj、cls

通道 12~17 : Anchor2 的 tx、ty、tw、th、obj、cls

实现逻辑：pixel_logits[0:17]接收通道数据。当 detect_channel_base==16 时，一个网格位置的 18 通道已经收齐，模块将其复制到 pending_logits，并保存检测头编号、网格坐标和帧号。后处理空闲时再转移到 active_logits。每个 Anchor 需要对 6 个 logit 执行 Sigmoid。模块使用 3 个并行 Sigmoid ROM，每个 Anchor 分两拍处理：第一拍处理 tx、ty、tw，第二拍处理 th、obj、cls。三个 Anchor 共使用 6 个 issue phase。

关键代码：

```
if (detect_valid) begin
```

```
    for (index = 0; index < 8; index = index + 1) begin
        if (detect_lane_valid[index] && (detect_channel_base + index < 18))
            pixel_logits[detect_channel_base + index] <=
                detect_logits_q6_10[index*16 +: 16];
    end
    if (detect_channel_base == 16) begin
        pending_logits[16] <= detect_logits_q6_10[15:0];
        pending_logits[17] <= detect_logits_q6_10[31:16];
        pending_valid      <= 1'b1;
    end
end
```

Anchor 尺寸如下：

检测头	特征图	stride	Anchor 尺寸
Detect0	80x48	4	(5,3)、(10,5)、(18,8)
Detect1	40x24	8	(24,10)、(40,16)、(72,28)

表 11 Anchor 尺寸

5.2 Sigmoid 查找表与边界框解码

yolo_sigmoid_lut.v

模块功能：将有符号 INT16 Q6.10 logit 转换为无符号 Q0.24 概率。ROM 包含绝对值 0~8192 对应的 8193 个表项，即覆盖 0~8.0；更大的绝对值被钳位到 8.0。负数通过 sigmoid(-x)=1-sigmoid(x)反射得到。yolo_postprocess.v 在五级寄存流水中完成 Anchor 解码、置信度计算和原图坐标映射。

实现逻辑：Sigmoid 查表逻辑为：

```
wire [15:0] magnitude = input_q6_10[15] ?
                        (~input_q6_10 + 1'b1) : input_q6_10;
wire [13:0] address = (magnitude > 8192) ? 14'd8192 : magnitude[13:0];
assign output_q0_24 = negative_d ? (25'h1000000 - table_data) : table_data;
解码公式与 YOLOv5 Detect 形式一致：
center_x = (2*sigmoid(tx) - 0.5 + grid_x) * stride
center_y = (2*sigmoid(ty) - 0.5 + grid_y) * stride
width    = (2*sigmoid(tw))^2 * anchor_w
height   = (2*sigmoid(th))^2 * anchor_h
score    = sigmoid(obj) * sigmoid(cls)
```

解码中间坐标使用带符号 Q24 表示，概率和置信度使用 Q0.24。置信度乘法结果加 2^23 后右移 24 位，实现舍入到 Q0.24。

关键代码：

```
center_x_q24_3 <= decode_head_2 ?
                          (center_x_unscaled_2 <<< 3) :
                          (center_x_unscaled_2 <<< 2);
candidate_score_3 <= (score_product_2 + 48'h000000800000) >> 24;
```

5.3 硬件 Top-K 排序

yolo_postprocess.v

模块功能：置信度不低于 0.25 的候选框进入 32 级流式比较交换链，最终保留置信度最高的 32 个候选框。

实现逻辑：阈值 0.25 在 Q0.24 下表示为 `24'h400000`。新候选从槽位 0 进入；若新分数严格大于当前槽位分数，则两者交换，被替换的数据继续向后传递，否则新候选直接流向下一槽位。比较使用严格大于号，因此相同分数时先到达的候选保持在前，排序结果具有稳定性，不需要单独的到达顺序计数器。sort_valid[31:0] 跟踪候选在 32 级链中的位置，sort_inflight 跟踪尚未排空的候选数。只有 CNN 结束、Sigmoid/解码流水线清空且 sort_inflight==0 后才启动。

关键代码：

```
localparam SCORE_THRESHOLD = 24'h400000;
wire sort_accept = decode_valid_5 &&
                          (candidate_score_5 >= SCORE_THRESHOLD) &&
                          (post_state == POST_COLLECT);
if (sort_valid[index]) begin
    if (sort_score[index] > top_score[index]) begin
        top_score[index] <= sort_score[index];
        if (index < 31)
            sort_score[index+1] <= top_score[index];
    end else if (index < 31) begin
        sort_score[index+1] <= sort_score[index];
    end
end
```

5.4 无除法器 NMS

NMS 同样集成在 yolo_postprocess.v。模块最多遍历 Top-32 候选，并最多保留 4 个框。每个新候选依次与已保留框比较；只要任一 IoU 超过 0.45，该候选就被抑制。

实现逻辑：状态机依次执行候选读取、宽高计算、面积乘法、阈值判断和保留/抑制操作。IoU 比较采用交叉相乘：

intersection / union > 0.45

等价于

intersection * 100 > union * 45

因此无需除法器，也无需 Q16 阈值乘法器。处理到 32 个候选、遇到空 Top 槽位或已经保留 4 个框时结束，并拉高 result_valid。

关键代码：

```
wire [31:0] intersection_x100 =
      ({8'd0, intersection_area_q} << 6) +
      ({8'd0, intersection_area_q} << 5) +
      ({8'd0, intersection_area_q} << 2);
wire [31:0] union_x45 =
      ({7'd0, union_area_q} << 5) +
      ({7'd0, union_area_q} << 3) +
      ({7'd0, union_area_q} << 2) +
      {7'd0, union_area_q};
wire suppress_candidate =
      (union_area_q != 0) && (intersection_x100 > union_x45);
if ((candidate_index == 32) ||
      (top_score[candidate_slot] == 0) ||
      (box_count == 4)) begin
      result_valid <= 1'b1;
      post_state      <= POST_HOLD;
end
```

6. 坐标映射与结果有效性

6.1 模型坐标映射到 1920x1080

坐标映射由 yolo_postprocess.v 中的 map_x_1080p 和 map_y_1080p 完成。解码坐标为 Q24 格式，映射结果是 12 bit 整数坐标，并在进入 Top-32 前完成。

实现逻辑：X 方向将 320 宽模型坐标乘 6，映射到 1920 宽原图。Y 方向先减去顶部 6 行填充，再乘 6 映射到 1080 高原图：

x_original = x_model * 6

y_original = (y_model - 6) * 6

X 坐标钳位到 0~1919，Y 坐标钳位到 0~1079。计算使用移位加法实现乘 6，并在右移 24 位前加入 2^23 完成整数舍入。

关键代码：

```
scaled = (value_q24 <<< 2) + (value_q24 <<< 1);
if (scaled < 0)
```

```
        map_x_1080p = 12'd0;
    else if (scaled > (64'sd1919 <<< 24))
        map_x_1080p = 12'd1919;
    else
        map_x_1080p = (scaled + 64'sh0000000000800000) >>> 24;
    padded = value_q24 - (64'sd6 <<< 24);
    scaled = (padded <<< 2) + (padded <<< 1);
    if (scaled < 0)
        map_y_1080p = 12'd0;
    else if (scaled > (64'sd1079 <<< 24))
        map_y_1080p = 12'd1079;
```

6.2 检测结果组织与有效标志

坐标在映射函数中已经完成边界钳位，NMS 通过 `box_count` 指示最终有效框数，并生成 4 bit box_valid。

实现逻辑：4 个框的每类坐标使用 48 bit 总线打包，每个框占 12 bit，低 12 bit 对应框 0。置信度使用 Q0.24，每框 24 bit。当前为单类别网络，所有框的 box_class_id 固定为 0。

关键代码：

```
assign box_valid = {(box_count > 3), (box_count > 2),
                    (box_count > 1), (box_count > 0)};
assign box_class_id = 32'd0;
assign box_x1 = {kept_x1[3], kept_x1[2], kept_x1[1], kept_x1[0]};
assign box_y1 = {kept_y1[3], kept_y1[2], kept_y1[1], kept_y1[0]};
assign box_x2 = {kept_x2[3], kept_x2[2], kept_x2[1], kept_x2[0]};
assign box_y2 = {kept_y2[3], kept_y2[2], kept_y2[1], kept_y2[0]};
```

7. 帧同步、结果显示与串口输出

7.1 跨时钟与握手

yolo_frame_mailbox.v

模块功能：使用请求/应答 toggle 协议传递已完成输入帧的 Bank 号和 16 bit 帧号。源侧在 source_frame_done 时发布任务，推理侧在 inference_job_done 时应答。

实现逻辑：多位 published_bank 和 published_frame_id 在发布后保持稳定，直到推理完成，因此属于 bundled-data 握手，而不是逐位同步。请求和应答 toggle 分别经过两级同步寄存器。当前工程两侧使用同一个 pixclk_in，该结构主要用于帧所有权管理，并为未来拆分时钟域保留 CDC 能力。若新帧完成时上一个发布任

务尚未应答，source_overrun 置位。推理完成条件不是 CNN 的 cnn_done，而是完整后处理结果 detector_result_valid，从而确保输入 Bank 在后处理结束前不会被重新发布。

关键代码：

```
if (!inference_job_valid && (publish_sync_2 != consumed_toggle)) begin
    inference_read_bank <= published_bank;
    inference_frame_id  <= published_frame_id;
    inference_job_valid <= 1'b1;
    consumed_toggle     <= publish_sync_2;
end

if (inference_job_valid && inference_job_done) begin
    inference_job_valid <= 1'b0;
    acknowledge_toggle  <= consumed_toggle;
end
assign inference_job_done = detector_result_valid;
```

7.2 串口输出

plate_uart_tx.v

模块功能：将最多 4 个检测框的轴对齐矩形坐标发送给 RK3568 或上位机。接口只输出 x1、y1、x2、y2，不发送四角点、置信度或类别。

实现逻辑：串口采用 8-N-1、LSB first 格式。顶层在约 148.5 MHz pixclk_in 下设置 BAUD_DIV=1289，对应约 115200 bit/s。send_trig 的上升沿启动发送，状态机依次经历 ST_IDLE、ST_LOAD、ST_START、ST_DATA、ST_STOP。

UART 帧为变长格式：

Byte 0　　: 0xAA

Byte 1　　: box_count，范围 0~4

后续数据 ： 每个框 8 字节，依次为 x1、y1、x2、y2

每个坐标为 uint16，小端序，低字节在前

总长度　　: 2 + box_count * 8 字节

当前协议没有 0xCC 帧尾。0 个框时只发送 AA 00，4 个框时发送 34 字节。tx_busy 表示发送状态，但当前顶层没有利用它对检测结果施加反压；结果触发之间通常远大于一帧 UART 发送时间。

关键代码：

```
if (send_trig && !send_trig_d)
```

```
        state <= ST_LOAD;
tx_buf[0] <= 8'hAA;tx_buf[1] <= (box_count > 3'd4) ? 8'd4 : {5'd0, box_count};
tx_buf[2] <= box_x1[7:0];
tx_buf[3] <= box_x1[15:8];
tx_buf[4] <= box_y1[7:0];
tx_buf[5] <= box_y1[15:8];
case (box_count)
        3'd0: frame_len <= 6'd2;
        3'd1: frame_len <= 6'd10;
        3'd2: frame_len <= 6'd18;
        3'd3: frame_len <= 6'd26;
        default: frame_len <= 6'd34;
endcase
```

2 性能参数

2.1 RK3568 侧误差测算

2.1.1. 模型训练概述

在车牌检测识别部分，系统包含两条 YOLO26 检测路线和一个独立训练的车牌识别模型：

- 整帧检测模型：采用 YOLO26n-Pose-P2 adapt-spatial 结构，网络输入为 512×288，用于 AI 模式下对 HDMI 整帧进行车牌定位。
- 串口 ROI 模型：采用 YOLO26n-Pose-P2 prune1-spatial 结构，将 FPGA 提供的 1～4 个候选 ROI 按固定槽位组成 320×160 画布，用于车牌框和四角点的精定位。
- 识别模型：采用卷积网络与 CTC 损失的端到端架构，输入 168×48 的车牌裁剪图，直接输出字符序列，无需逐字符分割。支持含省份简称、字母、数字及特殊标识的 78 个字符类别。使用 Adam 优化器，阶梯式学习率衰减。

两条检测路线均输出 P2～P5 四个尺度，识别 single_plate 和 double_plate 两类车牌，并预测 TL、TR、BR、BL 四个包含 x、y、visibility 的关键点。模型外以 FP32 完成反量化、Sigmoid、框和关键点解码及全局 Top-K，不执行 NMS；识别模型仍按原流程部署。

2.1.2 YOLO26 注意力模块改造

原 YOLO26 的 C2PSA 包含两个 PSABlock，内部 Self-Attention 需要 Q/K/V 投

影、矩阵乘法、Softmax 和 Transpose，在 RK3568 的 RKNN 计算图中容易产生 CPU 回退，影响纯 NPU 性能。

本项目保留 C2PSA 和 PSABlock 的残差、前馈结构，只将内部两个 Attention 替换为 RKNNSpatialMixer：V=Conv1×1(x)，y=Conv1×1(V+DWConv3×3(V))，用卷积与逐元素加法完成局部上下文融合。

初始化复用原 QKV 中 V 通道及 BN，并继承位置编码和输出投影权重。adapt 同宽度对比：参数量 2.742M→2.709M，卷积 MAC 1.2287G→1.2240G。改造重点是消除 Softmax、Attention 矩阵运算和 Transpose 回退；最终检查点均为 2 个空间混合器、0 个 Self-Attention 模块。

2.1.3 实验配置

硬件：NVIDIA GeForce RTX 4060 Laptop GPU（8GB 显存），16GB 系统内存

检测数据集：整帧路线为 21,011 张训练、4,690 张验证图像；ROI 路线为 25,688 张训练、5,568 张验证图像，包含 1～4 车牌及硬负样本。识别模型数据集保持原方案：CCPD2019，base、blur、challenge、db、rotate 五个子集中随机抽取 60,000 张训练，15,000 张验证。

参数	整帧检测	串口 ROI 检测	识别模型
训练轮数	60	80+15 轮 QAT	50
批大小	32	256（QAT 为 32）	64
初始学习率	0.001	0.01（QAT 为 0.0001）	0.001
优化器	MuSGD	MuSGD	Adam

表 12 RK3568 端模型训练超参数

检测模型训练与验证指标：

模型/阶段	Box mAP50	Box mAP50-95	Pose mAP50	Pose mAP50-95
512×288 FP32	0.9849	0.8307	0.9851	0.9838
320×160 FP32	0.9926	0.9185	0.9926	0.9925
320×160 QAT 假量化	0.9928	0.9190	0.9928	0.9927

表 13 检测模型训练与验证指标

整帧模型训练 60 轮、batch 32，采用 MuSGD、1 轮 Warmup 及线性衰减，最后

10 轮关闭 Mosaic。ROI 模型训练 80 轮、batch 256，采用 MuSGD 和 3 轮 Warmup，随后执行 15 轮 QAT；假量化结果与 FP32 基线基本一致。

识别模型训练指标：

轮次	训练损失	字符准确率	整牌准确率	学习率
1	2.874	0.423	0.186	0.001
5	1.052	0.712	0.498	0.001
10	0.519	0.836	0.672	0.0005
15	0.325	0.889	0.768	0.0005
20	0.218	0.918	0.831	0.0003
25	0.156	0.937	0.872	0.0003
30	0.113	0.951	0.899	0.0001
35	0.087	0.960	0.918	0.0001
40	0.070	0.967	0.932	0.0001
45	0.058	0.972	0.942	0.0001
50	0.050	0.976	0.950	0.0001

表 14 模型训练数据

训练损失从 2.87 快速收敛至 0.05，前 5 轮即完成主要学习过程。整牌准确率从 0.186 提升至 0.950，验证集指标与训练集同步收敛，未出现过拟合现象。

2.1.4 模型的量化与部署

量化流程：

检测模型量化流程：512×288 整帧模型采用 W8A8 PTQ INT8；320×160 ROI 模型采用 QAT 混合精度方案，主干、颈部和预测前级使用 W8A8，最终 12 个预测卷积及 P2～P5 四路输出保持 FP16。

识别模型量化流程保持原方案：量化过程使用 100 张从验证集中随机抽取的校准图像，覆盖不同光照、角度和模糊场景。Rknn-Toolkit2 工具根据校准数据的激活值分布确定每层的量化参数，将浮点权重和激活值映射到 INT8 范围，优化级别设为最高级 Level 3。

检测模型的反量化、Sigmoid、边界框与关键点解码及全局 Top-K 均在模型外以 FP32 执行，且不使用 NMS。该设计既避免动态后处理算子进入量化图，也便于板端调整置信度阈值。

检测模型量化验证：

模型	指标	FP32	量化验证	差值
512×288	Box mAP50-95	0.8307	未实测	—
512×288	Pose mAP50-95	0.9838	未实测	—
320×160	Box mAP50-95	0.9185	0.9190（QAT）	+0.0005
320×160	Pose mAP50-95	0.9925	0.9927（QAT）	+0.0002

表 15 检测模型量化验证

识别模型量化精度对比：

模型	指标	ONNX	RKNN	INT8 精度损失
识别	字符准确率	0.976	0.968	-0.008
识别	整牌准确率	0.950	0.936	-0.014

表 16 识别模型量化精度对比

320×160 模型的 QAT 假量化验证未出现总体精度下降；512×288 模型仅有 FP32 完整验证和板端 INT8 耗时，尚无同口径 RKNN 精度报告，故不填写量化损失。识别模型整牌准确率下降 1.4 个百分点，处于可接受范围；其纯卷积结构数值稳定，量化适配性较好。

推理性能对比：

在 RK3568 开发板上实测的单帧推理耗时：

处理阶段	ONNX	RKNN
整帧检测推理	未记录	Mean 17.81ms，P95 18.03ms
串口 ROI 检测推理	未记录	Mean 7.21ms，P95 7.63ms
识别推理	30ms	1ms

表 17 RK3568 实测单帧推理耗时

RK3568 实测表明，两条检测路线的纯 NPU 耗时均低于 20ms，其中 ROI 模型更适合 FPGA 已提供候选区域的串口模式。识别推理仍由 30ms 降至 10ms，满足实时部署要求。

本地实际遍历 CBLPRD-330k 数据集 300 张车牌（蓝 100、绿 100、黄 90、白 5、黑 5）准确率在 97%，主要集中在蓝黄颜色识别的准确率上，由于数据集本身图像质量的限制大约五张出现完全识别不到，约有 5 张样本出现颜色误判或字符识别错误。

2.2. FPGA 资源开销

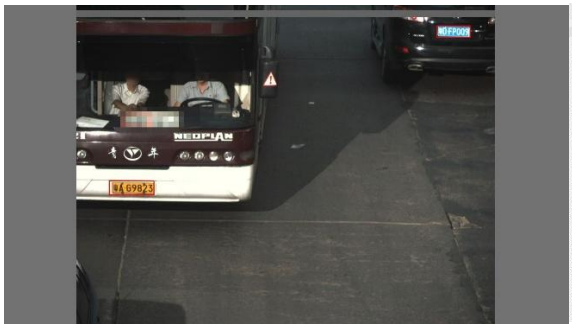
Resource Usage Summary

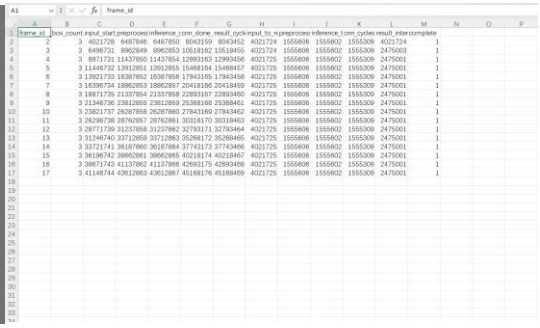
Find:

	Logic Utilization	Used	Available	Utilization(%)
1	APM	89	740	13
2	FF	16135	319600	6
3	LUT	12699	159800	8
4	Distributed RAM	320	39500	1
5	DRM	301	415	73
6	IO	137	400	35
7	RCKB	2	40	5
8	SCANCHAIN	0	1	0
9	USCM	4	32	13
10	CCS	1	1	100
11	ADC	0	1	0
12	DDR_PHY	7	40	18
13	HSSTLP	0	2	0
14	GPLL	4	10	40
15	PPLL	2	10	20
16	DDRPHY_CPD	1	20	5
17	HCKB	0	120	0
18	IOCKB	0	40	0
19	MRCKB	1	20	5
20	PCIE	0	1	0
21	DDRPHY_IOCLK_DIV	2	10	20
22	ANALOG	0	1	0
23	TSERDES	14	80	18

2.3. FPGA 车牌识别误差分析

使用 Verilator 对真实车牌图像、HDMI 1080p60 输入、双 Bank 输入缓存、Mailbox 帧级握手、9 层 CNN、后处理及视频叠加链路进行联合仿真。测试连续验证 16 帧，CNN 均正常完成检测，无丢帧或帧序错乱。每帧稳定输出 2 个轴对齐车牌候选框：(1470,70)-(1585,109)，置信度约 0.872922；(357,596)-(509,643)，置信度约 0.872055。检测结果与 PTQ 整数参考数据逐项一致，检测有效标志均正常，未出现 Mailbox 数据丢失、缓存溢出、检测结果错误或视频叠加缺失。





3. 仿真与测试报告

3.1. 系统模块功能仿真测试

仿真时为验证重要模块功能时序与逻辑的正确性稳定性，同时避开未开源 IP 的逻辑建模，对 FPGA 工作的主要难点重点进行仿真展示。对 YOLO 的 FPGA 工程完成，仿真 MAC 与重量化、图像预处理，实际测试 RTL 及测试功能如下：

模块	实际测试 RTL	定量输入	验证功能
MAC 流水线	yolo_mac16x8_vivado_tb.v yolo_mac16x8.v yolo_dot4_u8_i8	固定 16×8 向量	MAC 结果、流水线延迟
图像预处理	yolo_preprocess_6x6_1080p_vivado_tb.sv yolo_preprocess_6x6.v	1920×1080 RGB 帧	6×6 降采样、边界填充、地址输出
重新量化	yolo_requantize_8_tb.sv yolo_requantize_8.v	2073600 组 8 路数据	舍入移位、ReLU、数据饱和

表 18 仿真模块功能

3.2 仿真环境与参数设置

1. PDS 与 ModelSim SE-64 10.4 联合仿真。
2. 仿真环境

- 仿真工具：Verilator
- RTL 语言：Verilog/SystemVerilog
- 辅助语言：Python、C++
- 输入数据：模型权重、测试图像、Golden 中间张量
- 验证方法：RTL 输出与软件参考结果逐项比较
- 模块级仿真

3. Vivado 2022.2

3.3 模块仿真结果分析

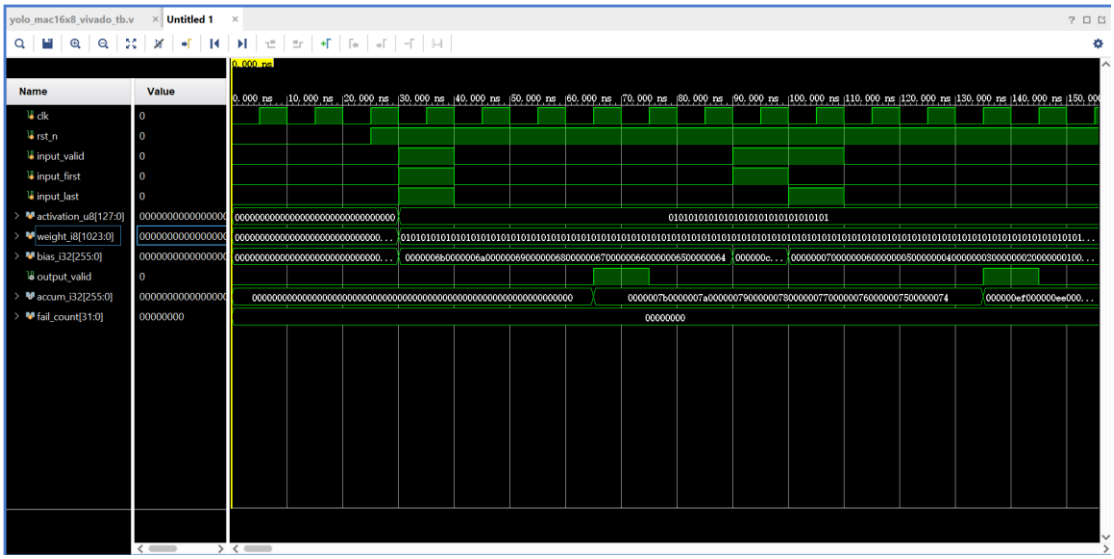
1. 16×8MAC 仿真分析

yolo_mac16x8

仿真模块功能：用于完成 8 个输出通道的并行乘加计算。模块每次接收 16 个 UINT8 激活值、对应的 INT8 权重和 8 路 INT32 bias，通过四组 dot4 乘法单元完成点积，再经过两级加法树和累加器得到 8 路 INT32 输出。模块同时对 input_valid、input_first 和 input_last 进行三级流水同步，支持连续时钟输入和分段累加。

仿真逻辑：testbench 产生周期为 10 ns 的时钟，并在开始阶段保持低有效复位信号，复位释放后首先发送一个 input_first=1、input_last=1 的完整请求。仿真中将全部激活值和权重设置为 1，因此每个输出 lane 的点积结果为 16，bias 设置为 100～107，用于验证单次 bias 加点积计算。随后连续发送两个相邻请求，第一个请求设置为 first=1、last=0，第二个请求设置为 first=0、last=1，验证流水线连续输入、first/last 控制信号延迟以及跨请求累加功能。testbench 对 output_valid 延迟、8 个 lane 的 accum_i32 结果和 fail_count 进行自动比较。

仿真结果：仿真结果符合预期。第一次请求每个 lane 的输出分别为 116～123，即 bias 加上 16 点积结果；第二组连续请求完成后，每个 lane 在原有结果上再次增加 16，最终输出为 232～239。波形中可以看到输入请求经过三级流水后产生 output_valid，且输出有效信号只保持一个时钟周期。fail_count 始终为 0，说明乘加计算、bias 加法、连续累加、流水延迟以及输出握手协议均验证通过。



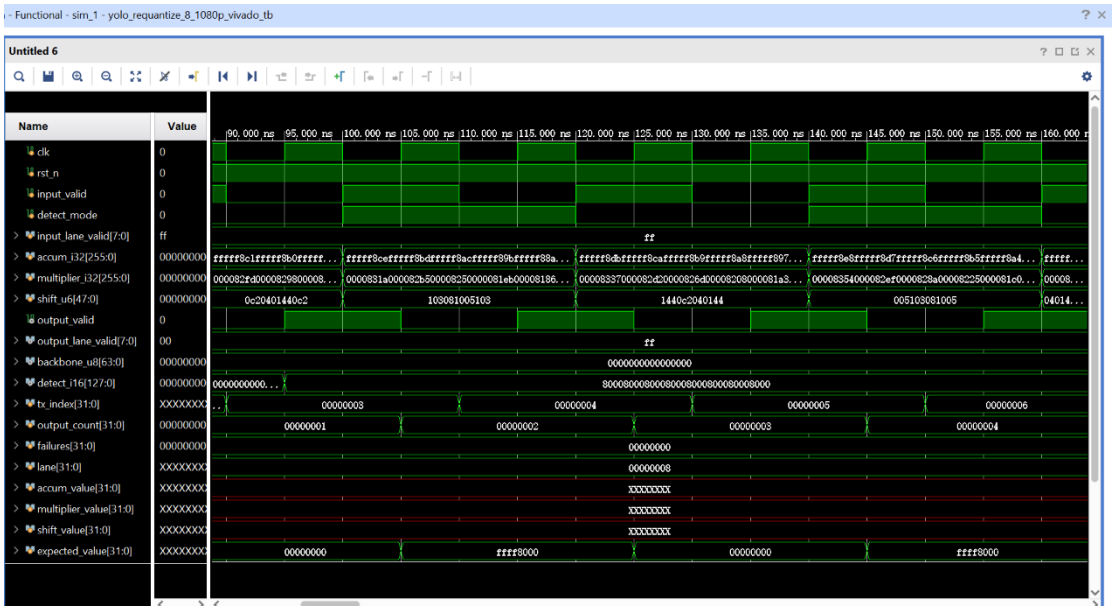
2. 重量化模块仿真

yolo_requantize_8

仿真功能：本仿真用于验证 yolo_requantize_8 对 1920×1080 全帧规模数据的重新量化功能。模块接收 8 路 INT32 累加结果、8 路乘法系数和移位参数，根据 detect_mode 分别生成 Backbone UINT8 输出或 Detect INT16 输出，同时检查舍入、算术右移、ReLU、UINT8 饱和以及 INT16 饱和行为。

仿真逻辑：testbench 在复位释放后，按照像素顺序发送 2,073,600 组八通道输入数据，每组输入包含独立生成的累加值、乘法系数和移位量，并交替切换 Backbone 与 Detect 模式。testbench 根据相同的定点计算规则建立参考结果，在 output_valid 有效时检查 output_lane_valid、8 路输出数据及输出顺序，从而验证流水线延迟和连续输入条件下的参数同步是否正确。

仿真结果：全帧规模仿真完成后，模块能够连续输出全部 2,073,600 组结果，output_lane_valid 保持为 8'hFF，Backbone 模式下输出正确执行 UINT8 饱和和 ReLU，Detect 模式下输出正确执行有符号 INT16 饱和，所有输出与参考模型逐 lane 完全一致，最终 output_count=2,073,600、failures=0，Transcript 中显示：PASS: yolo_requantize_8 1920x1080 full-volume test。



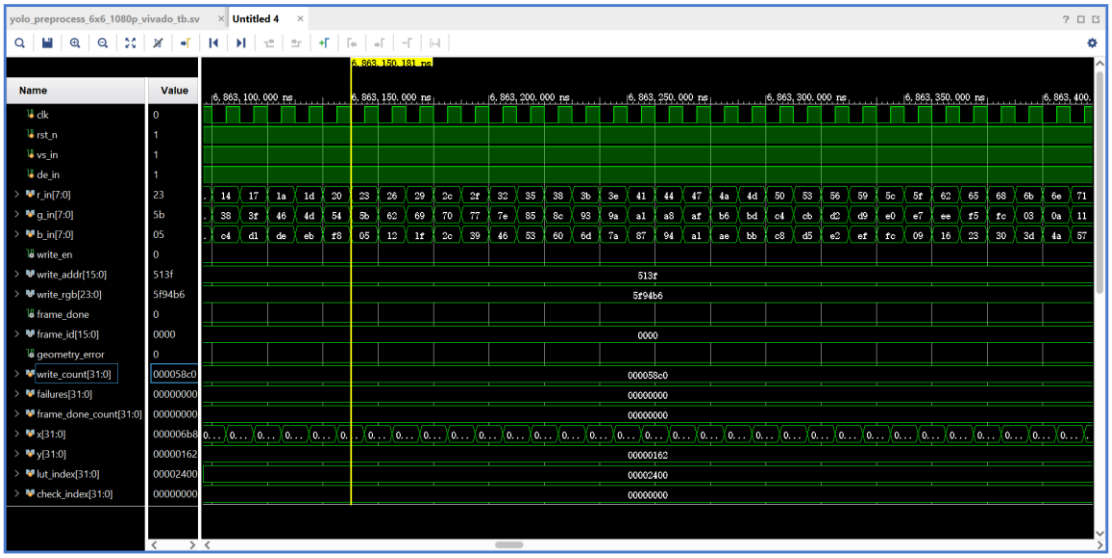
3. 预处理仿真

yolo_preprocess_6x6

仿真功能：对完整 1920×1080 RGB888 视频帧的预处理功能。模块将输入图像按照 6×6 像素块进行降采样，将 1920×1080 图像转换为 320×180 内容图像，并在上下各增加 6 行灰色填充，最终生成 320×192、共 61440 个输出像素。仿真同时检查输出地址、RGB 像素值、帧号、帧完成标志和几何错误标志。

仿真逻辑：testbench 首先对 DUT 施加复位，复位释放后通过 vs_in 的下降沿启动一帧，并按照 1920×1080 的光栅顺序逐像素产生 r_in、g_in、b_in 和 de_in。DUT 每累计 6×6 个输入像素后，通过除以 36 的查找表完成通道平均，并依次输出 RGB 数据；仿真记录每次 write_en 有效时的 write_addr 和 write_rgb，最后检查顶部填充、底部填充、全部 320×180 个内容像素以及最终 frame_done 和 frame_id。

仿真结果：仿真结果表明，yolo_preprocess_6x6 能够正确接收完整的 1920×1080 RGB888 视频帧，并按照 6×6 像素块完成降采样，生成 320×180 的有效图像内容，同时在上下两侧正确插入 6 行填充数据，最终输出 320×192、共 61440 个像素。仿真对所有输出像素的地址、RGB 数据、填充区域、帧号和帧完成标志进行了检查，结果中 geometry_error=0、failures=0、write_count=61440、frame_done_count=1，且 frame_id 正确更新为 1，说明该模块的图像缩放、边界填充和逐帧输出协议均满足设计要求。



4. Verilator RTL 仿真

通过 Verilator RTL 仿真，测试采用真实车牌图像，联合验证 HDMI 视频输入、双 Bank 帧缓存、Mailbox 帧级握手、9 层 CNN、检测后处理及车牌框叠加链路。连续 16 帧均完成检测且每帧稳定输出 3 个车牌框，无丢帧、帧序错误、缓存溢出或叠框缺失。CNN 单帧计算周期为 1,555,308 周期，在 148.5 MHz 下约为 10.47 ms，对应核心吞吐率约 95.48 次/s；最大端到端延迟为 4,021,725 周期，约 27.08 ms。虽然单帧结果存在约 27 ms 延迟，但 CNN 计算时间小于 16.67 ms 的视频帧周期，连续检测结果间隔稳定为 16.67 ms，因此系统可持续处理 1920×1080@60 Hz 视频流。

4. 总结

4.1 可扩展之处

当前 YOLO FPGA 工程以 HDMI 输入的 1920×1080 RGB 图像为输入，首先通过 yolo_preprocess_6x6 完成 6×6 像素块降采样，将图像转换为 320×180 内容区域，并加入上下边界填充，生成 320×192 的网络输入特征图。随后由 yolo_cnn_engine 完成卷积层调度、特征图缓存和检测流程，卷积计算采用 yolo_mac16x8 输出 16×8 MAC 阵列，通过多级流水线将激活值、权重、乘法结果和加法树进行寄存器切分。MAC 输出经 yolo_requantize_8 完成 INT32 到 UINT8 或 INT16 的重新量化，并实现舍入、移位、ReLU 及饱和处理。检测头完成目标框解码和坐标映射后，通过 plate_uart_tx 按固定帧格式发送车牌坐标，

实现从图像预处理、CNN 推理、检测结果解码到坐标输出的完整硬件流程。

1. 图像预处理与光照适应方面，可在现有 Gamma 查找表基础上增加亮度统计模块，根据帧均值、直方图或局部亮度动态选择 Gamma 参数，实现自适应 Gamma 校正。还可以进一步加入同态滤波、白平衡、对比度拉伸和局部直方图均衡等模块，提高逆光、夜间、强反光和阴影条件下的图像质量。相关算法可采用查找表、行缓存和流水线结构实现，以减少对实时性的影响。
2. CNN 网络和硬件计算方面，可在现有 YOLO 网络及 yolo_cnn_engine 基础上继续优化网络结构，增加多尺度特征融合，提高系统对远距离、小尺寸和倾斜车牌的检测能力。同时可以扩展车牌颜色、车牌类型和字符识别分支，使系统由单一车牌检测逐步升级为“车牌定位、透视校正、字符识别和颜色分类”的完整流水线。在硬件实现方面，可以继续优化 yolo_mac16x8 的流水线级数、BRAM 乒乓缓存和卷积引擎复用策略，并研究剪枝、知识蒸馏、混合精度量化和稀疏卷积，在保持检测精度的同时降低 BRAM、DSP 和计算周期消耗。

4.2 心得体会与小组分工

通过本项目的开发，我们认识到，面向实时车牌检测的 FPGA+ARM 异构系统并不是单一的 Verilog 编程或神经网络部署问题，而是算法设计、硬件架构、存储管理、数据传输和系统调试的综合工程。决赛阶段，我们围绕 1920×1080 HDMI 视频输入、YOLO 推理、检测框坐标映射及 UART 输出构建了完整的数据链路，并通过模块级仿真、逐周期波形分析、定点结果对比和上板验证，重点排查了跨时钟域传输、复位、帧同步、流水线延迟和 CNN 控制信息同步等问题，逐步形成了较完整的 FPGA 开发与验证流程。

从初赛到决赛团队分工明确、协作紧密：一名队员负责 RK3568 端检测与识别模型的训练、量化和部署，完成 PC 端上位机开发及 RK3568 与 FPGA 之间的通信实现；一名队员负责 FPGA 车牌检测 CNN 的网络结构设计、数据集处理、模型训练、量化验证和部署参数导出，视频流接口串口等硬件实现；一名队员负责 CNN 网络的 FPGA 全流程部署，完成 INT8 卷积加速、量化、解码、存储调度及视频系统集成，并联合开展网络结构与硬件资源优化。团队共同完成模型精度、硬件资源、计算延迟和系统吞吐率之间的协同优化。

5 参考文献

[1]深圳市紫光同创电子有限公司. Logos2 系列 FPGA 器件数据手册：DS04001 V1.5[S]. 2020.

[2] OmniVision Technologies Inc. *OV5640 datasheet: 1/4-Inch color CMOS QSXGA (5 Megapixel) image sensor with OmniBSI™ technology* (version 2.01) [EB/OL]. (2011-01) [2024-11-20]. https://www.ovt.com/sensors/OV5640.

[3] 深圳市小眼睛科技有限公司. MES-FMC-LPC-IO 子卡硬件使用手册[M]. v1.1. 深圳: 深圳市小眼睛科技有限公司, 2025.

[4]深圳市小眼睛科技有限公司. MES2L676-200HP 硬件使用手册[M]. v1.2. 深圳: 深圳市小眼睛科技有限公司, 2024.

[5] S. Chen, Q. Song, H. Guo, and X. Li, "Design of license plate recognition system based on FPGA," in *2022 2nd Asia-Pacific Conference on Communications Technology and Computer Science (ACCTCS)*, Shenyang, China, 2022, pp. 227-231. doi: 10.1109/ACCTCS53867.2022.00054.

[6]正点原子仓库地址：https://github.com/alientek-openedv/Products.git

6 AI 使用说明

本团队在系统的研发与文档撰写过程中，严格遵守学术诚信与比赛规范。为了提高研发效率和突破技术瓶颈，我们在部分环节合理地借助了 AI 工具的辅助。特此声明本作品中 AI 工具的具体使用情况如下：

一、 创意与方案辅助举例

使用的 AI 工具：ChatGPT / Gemini

1. 查找表数据的离线生成（分赛区决赛）

涉及文件： sigmoid_center.hex、sigmoid_corners.hex、sigmoid_offsets.hex

320×320 网络的检测 Head 需要输出角点置信度与亚像素偏移，算法端依赖 Sigmoid 激活函数。FPGA 硬件无法直接计算指数运算，因此采用查找表（LUT）

方案：将 Sigmoid 函数离散化为 256 个采样点，预先计算定点值并导出为.hex 文件，供 FPGA 的 BRAM ROM 初始化使用。

AI 辅助编写了 Python 脚本，用于在离线环境下批量计算各采样点的 8 位定点量

化值，并输出为十六进制格式。

说明： 量化位宽（8 位输入/8 位输出）和采样点数（256 点）由团队成员根据硬件资源与推理精度要求确定。AI 仅协助编写数据生成脚本，**不涉及任何**硬件设计。FPGA 端的 BRAM 例化和查表控制逻辑由团队成员手工 RTL 实现。

2. Q9.7 定点数格式的数学分析

涉及场景： 坐标解码中的位宽分配与数值范围分析

在 320×320 网络的坐标解码过程中，粗坐标、局部峰值、精细偏移三者需要合成为最终的 Q9.7 格式角点坐标。设计初期，团队对整数部分 9 位是否能覆盖 1920×1080 的显示范围、小数部分 7 位的精度是否满足亚像素定位需求存在疑虑。AI 帮助团队分析了 Q 格式定点的数值范围与精度：9 位整数部分可覆盖 0~511，足以覆盖 1920×1080；7 位小数部分精度为 1/128≈0.0078 像素，满足亚像素级定位需求。基于该数学分析结论，团队成员独立完成了 Verilog 中的位移拼接逻辑和符号扩展编码。

说明： AI 提供的帮助仅限于定点数格式的数值范围分析，属于数学层面的理论推演。所有的硬件实现细节（位拼接、符号扩展、流水线划分）均由团队成员自主完成。

2. 色彩阈值调试工具**（初赛阶段，已废弃）**

涉及文件： Python GUI 调节器脚本

在初赛阶段的颜色检测方案中，团队需要确定 YCbCr 色彩空间中车牌颜色的有效阈值范围。AI 辅助编写了一款轻量级 Python GUI 调节器，通过滑动条实时调节各通道阈值并同步显示二值化效果。该工具加速了阈值标定过程，最终确定的参数被固化在 FPGA 的色彩状态机中。

说明： 该工具仅用于参数可视化标定，不涉及任何硬件逻辑。

二、 代码调试与验证辅助举例

使用的 AI 工具：ChatGPT / Claude

1. 仿真波形分析与问题定位辅助

在模块级仿真验证过程中，某些信号行为与预期不符时，团队需要从仿真波形中抓取关键信号分析时序关系。AI 辅助分析了波形中特定信号跳变的因果关系，帮助团队理解信号之间的时序依赖。

说明： 问题定位和代码修改均由团队成员完成，AI 仅辅助解释波形信号之间的因果关系。所有的调试结论由团队成员基于对设计意图的理解独立做出。

2. Testbench 通用框架模板

在编写仿真测试激励时，AI 辅助生成了部分 Testbench 的通用框架代码（时钟生成、复位控制、模块例化等重复性模板）。具体的信号赋值、时序控制、预期值判断和覆盖率规划均由团队成员根据模块功能需求独立编写。

说明： AI 生成的仅为通用的空壳模板，不包含任何与 CNN 功能相关的测试逻辑。所有测试用例的设计均源于团队成员对模块行为的理解。

三、 文档撰写与语言润色举例

使用的 AI 工具：ChatGPT /豆包大模型

团队成员在撰写技术报告初稿时，表述较为口语化。我们将核心逻辑要点交由 AI 润色，AI 帮助规范了专业术语，使复杂硬件时序逻辑的描述更加符合学术严谨性要求。

说明： 所有技术内容、架构分析、设计思路均为团队原创，AI 仅对表达方式进行规范化调整，不产生任何技术结论。

四、 AI 生成内容占比申明

内容类别	**AI 辅助占比**	**团队原创占比**
RTL 代码（Verilog .v 文件）	0%	100%
硬件架构设计与状态机	0%	100%
模型训练与量化参数提取	0%	100%
综合约束与布局布线	0%	100%
上板调试与验证	0%	100%
技术文档正文	约 8%（语言润色）	92%
设计思路与技术结论	5-10%	90-95%
查找表数据文件（.hex）	约 10%（批量生成脚本）	90%
测试数据与实验分析	5%	95%
Python 调试工具	约 10%（框架代码）	90%
Testbench 通用模板	约 8%（时钟/复位框架）	92%

表 19 AI 生成内容占比说明